

Duale Hochschule Baden-Württemberg Mannheim

Bachelorarbeit

**Evaluierung und Konzeption von Web Components zur skalierbaren Einbindung
von Cloud Anwendungen im E-Commerce Umfeld**

Studiengang Wirtschaftsinformatik

Studienrichtung Data Science

Sperrvermerk

Verfassende Person:	Alisa Rogner
Matrikelnummer:	7894464
Firma:	Schaeffler Technologies AG & Co.KG
Abteilung:	Custom Data and AI 2
Kurs:	WWI20DSB
Studiengangsleitung:	Prof. Dr. Bernhard Drabant
Wissenschaftliche Betreuung:	Simon Poll simon.poll@sap.com +4962277-65679
Firmenbetreuung:	Robert Krause krausrbe@schaeffler.com
Bearbeitungszeitraum:	13.02.2023 - 08.05.2023

Ehrenwörtliche Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit mit dem Thema: *Evaluierung und Konzeption von Web Components zur skalierbaren Einbindung von Cloud Anwendungen im E-Commerce Umfeld* selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Ort, Datum

Alisa Rogner

Sperrvermerk

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anders lautende Genehmigung der Ausbildungsstätte vorliegt.

Kurzfassung (Abstract)

Titel: Evaluierung und Konzeption von Web Components zur skalierbaren Einbindung von Cloud Anwendungen im E-Commerce Umfeld

Verfassende Person: Alisa Rogner

Kurs: WWI20DSB

Ausbildungsstätte: Schaeffler Technologies AG & Co.KG

Um die Integration der von Schaeffler selbst entwickelten Engineering Apps in das E-Commerce Portal Medias mit einer neuen Technologie, anstatt der mit Problemen behafteten iFrames zu lösen, wurde im Zuge dieser Arbeit das iFrame mit alternativen Möglichkeiten zur Integration verglichen. Dabei wurde entschieden, als neues Lösungsmuster eine Web Component zu nutzen, die dafür sorgen soll, dass die Anwendung immer auf dem neusten Stand ist, Nutzeranalysen einfacher werden und dass in die App leichter responsive Verhaltensweisen implementiert werden können. Im Zuge dessen wurde eine Reihe praktischer Tests durchgeführt, um zu überprüfen ob die Web Component, abgesehen von ihren theoretischen Vorteilen, auch für den Use Case geeignet ist. Dazu wurde eine Testanwendung entworfen, welche das Verhalten der Web Component in einem geschützten Raum überprüfen sollte, ohne die Engineering App zu beeinflussen. Im Zuge der Tests wurden die bereits in der Theorie identifizierten Vorteile wie ein natives Browserverhalten bestätigt, und schließlich Herausforderungen für die Implementierung herausgestellt, für die wiederum mehrere Lösungsmuster auf ihre Tauglichkeit verglichen wurden, um ihren Nutzen zu prüfen. Schließlich wurde das gefundene Wissen auf die Engineering App angewendet, indem diese zu einer funktionierenden Web Component umgebaut wurde, die auch produktiv in die Mediasumgebung eingebunden wird. Zum Schluss wurden schließlich noch Möglichkeiten zur Weiterentwicklung herausgestellt, die in nachfolgenden Sprints der Abteilung, sowie möglicherweise auch im Zuge einer weiteren Studentenarbeit bearbeitet werden können.

Inhaltsverzeichnis

Kurzfassung (Abstract)	iii
1 Einleitung	1
1.1 Hintergrund und Motivation	1
1.2 Zielsetzung	1
1.3 Vorgehensweise	2
2 Theoretische Grundlagen	3
2.1 Definition von E-Commerce Plattformen	3
2.2 Definition von Micro Frontend Frameworks	5
2.3 Überblick über die schaefflereigenen Engineering Apps	7
3 Vorgefundene Lösungsmuster mit Vergleich	11
3.1 Kriterien zur Auswahl der Lösungsmuster	11
3.2 Grundlagen von iFrames	12
3.3 Grundlagen von Web Components	14
3.4 Überblick und Vergleich der gefundenen Lösungsmuster anhand der von Engineering Apps und Medias gesetzten Bedingungen	15
4 Ansatz zur Nutzung der Web Component	18
4.1 Praxistest der Web Component als Lösungsmuster	18
4.2 Identifikation von Herausforderungen bei der Implementierung	23
4.2.1 Datei-Caching der benötigten Skripte	24
4.2.2 Mischung der Styles von Medias und Schaeffler	27
4.3 Konzept der Einbindung	31
5 Umsetzung der Engineering App als Web Component	33
5.1 Umbau der Engineering App zur Web Component	33
5.2 Anpassungen der Jenkins Pipeline	34
5.3 Style Isolation der Web Component	38
5.4 Weitere Anpassungen an der Engineering App	39
5.5 Dokumentationen zur Einbindung der Web Component	41
5.6 Evaluierung der Webcomponent	41
5.7 Möglichkeiten zur Weiterentwicklung der Web Component für die Enginee- ring App	45

6	Abschlussbetrachtung	46
6.1	Ergebnis	46
6.2	Ausblick	47
	Anhang: Vision der Engineering Apps	49
	Literatur	50

Abbildungsverzeichnis

Abbildung 2.1	E-Commerce Umsatz Weltweit (Bildquelle: [17, Seite 4])	4
Abbildung 2.2	Abgrenzung zwischen E-Commerce und E-Business (Bildquelle:[14, Seite 5])	5
Abbildung 2.3	Kooperation zwischen Bearingx und CDAI2	7
Abbildung 2.4	Benutzeroberfläche der Grease App	9
Abbildung 2.5	Benutzeroberfläche der Engineering App	10
Abbildung 4.1	Initialisierung eines Angular Projektes	19
Abbildung 4.2	Ordnerstruktur einer Angular App	20
Abbildung 4.3	Test Web Component in der Browseransicht	22
Abbildung 4.4	Konzept für die „ea-webcomponent“ Datei	26
Abbildung 4.5	Style Mischung der Parent Seite zur Web Component	27
Abbildung 4.6	Beidseitige Style Mischung	28
Abbildung 4.7	Aufbau einer Shadow DOM (Bildquelle: [24])	29
Abbildung 4.8	Style Mischung bei Verwendung einer Shadow Dom	29
Abbildung 4.9	Lösung des Styleproblems mittels einem Tailwind Prefix	30
Abbildung 4.10	Übersicht über die Dateienverteilung	31
Abbildung 5.1	Organisationsstruktur der Deployment Pipeline	35
Abbildung 5.2	Einbindung der Engineering App in Medias mittels iFrame	43
Abbildung 5.3	Einbindung der Engineering App in Medias mittels Web Component	44

Tabellenverzeichnis

Tabelle 3.1 Vorteile der Lösungsansätze	16
Tabelle 3.2 Nachteile der Lösungsansätze	17

Programmlistings

3.1 Einbindung einer Anwendung mittels iFrame (Codequelle: [8])	13
3.2 Einbindung einer Anwendung mittels Web Component (Bildquelle: [29])	14
4.1 Angular Installation	19
4.2 Initialisierung eines Angular Projekts	19
4.3 Initialisierung der Test Web Component	21
4.4 Initialisierung eines Web Servers (Codequelle: [20])	21
4.5 Einbindung der Web Component in eine HTML Seite	22
4.6 Verwendung des Parameters „Headline Input“	23
4.7 Initialisierung von Tailwind in der Globalen CSS Datei	28
5.1 Umbau der Engineering App zur Web Component	33
5.2 Setzen der Host URL in den verschiedenen Deployment Umgebungen	35
5.3 Erstellung der Platzhalter für die Javascript Dateien	36
5.4 Erstellung des Platzhalters für das Stylesheet	36
5.5 Abfrage der Java-Skript Dateien mithilfe von Regular Expressions	37
5.6 Ersetzung der Skripte innerhalb der Datei „ea-webcomponent.js“	38
5.7 Verwendung des Parameters „deploy-url“	40
5.8 Integration der Web Component in Medias	41

1 Einleitung

1.1 Hintergrund und Motivation

Als produzierendes Unternehmen ist es bei Schaeffler üblich, viele der entworfenen IT Anwendungen auch innerhalb des Unternehmens selbst zu nutzen. In einigen Fällen jedoch, werden die in Kooperation zwischen den Schaeffler Abteilungen entworfenen sogenannten Engineering Apps in externe E-Commerce Plattformen wie das von Schaeffler verwendete Medias eingebunden. Dies führt dazu, dass die Anwendungen dort auch von den Endkunden genutzt werden können. Das kann jedoch zu Herausforderungen führen, wenn die Einbindung der App nicht an die Anforderungen des Kunden angepasst ist. Wegen der derzeitigen Einbindung durch iFrames sind die Engineering Apps nur durch starken Zusatzaufwand responsive und passen sich nicht an das Zielformat der Seite an. Auch User Tracking ist aufgrund von Gegebenheiten der iFrame Technologie nur mit zusätzlichem Aufwand in Form eines zusätzlichen Google Analytics Skriptes umsetzbar. Eine Versionierung der Apps ist zusätzlich auch nicht gegeben, wodurch sich Updates zunehmend schwierig gestalten und ein Überblick über den Stand der Anwendung oft fehlt. Die bereits genannten Herausforderungen führen dazu, dass die ins Kundenportal eingebundenen Apps über Einschränkungen in der Darstellung verfügen, die auch für den Kunden zu sehen sind.

1.2 Zielsetzung

Ziel der Arbeit ist es, geeignete technische Maßnahmen zur Bereitstellung der von der Abteilung entwickelten Engineering Apps, wie den sich in der Entwicklung befindenden CO2 Calculator oder die bereits fertiggestellte Grease App, für E-Commerce Plattformen zu evaluieren. Hierfür ist vor allem die Plattform Medias relevant. Dabei sollen Alternativen zu den verwendeten iFrames gefunden werden. Eine mögliche Art der Einbindung, die dabei beleuchtet werden sollen, sind unter anderem eine Einbindung der App mittels einer Web Component, da dies die von Medias gewünschte Lösung ist. Für diese und andere Alternativen sollen daraufhin die Vor- und Nachteile gegeneinander abgewogen werden, was als Entscheidungsgrundlage für eine Variante zur Einbindung dienen soll. Sobald eine geeignete Einbindungsmethode identifiziert ist, sollen Herausforderungen bei der Implementierung dieser Methode gefunden und gelöst werden, und mit der Implementierung begonnen werden. Die Methode zur Einbindung soll am Ende auf die bereits existierende Engineering App angewendet werden, um diese in Medias einzubinden. Dadurch sollen auch Updates durch eine Versionierung der Apps vereinfacht werden, um eine skalierbare, problemlose Integration der

Engineering Apps in Medias gewährleisten zu können. Am Ende soll das Projekt in Bezug auf den geleisteten Benefit für Schaeffler evaluiert werden.

1.3 Vorgehensweise

Das geplante Vorgehen besteht daraus, dass zuerst durch eine Literaturrecherche Möglichkeiten zur Einbindung von Apps in E-Commerce Plattformen identifiziert werden sollen. Auf Basis derer werden Vor- und Nachteile der Lösungsmethoden herausgestellt. Die vielversprechendste Methode soll schließlich einem Praxistest mit einer kleinen Anwendung unterzogen werden. Dies soll dazu dienen, abwägen zu können, ob die gefundene Methode auch auf den von der Abteilung gestellten Use Case anwendbar ist oder ob eine Alternative gewählt werden muss. Die Anwendung, die nach den Analysen am sinnvollsten erscheint, soll letztendlich für die Einbindung der Engineering Apps in Medias gewählt werden. Im Zuge dessen wird ein Konzept entwickelt, wie sich diese Methode zur Einbindung realisieren lässt und die verschiedenen Herausforderungen bei der Anwendung sollen identifiziert werden. Gleichzeitig sollen auch potenzielle Lösungsmethoden für die Herausforderungen identifiziert werden. Das gewählte Konzept soll schließlich praktisch umgesetzt werden. Dabei ist es das Ziel, die App selbst mit möglichst wenig Aufwand so anzupassen, dass eine skalierbare und responsive Einbindung in Medias geschieht, die immer den aktuellsten Stand der App abbildet und den geringstmöglichen Aufwand für die Medias Entwickler darstellt. Daraufhin wird die Methode zur Einbindung evaluiert, um Fehler frühzeitig zu eliminieren und zu sehen, ob ein Benefit für Schaeffler generiert wird. Die eingebundene Anwendung soll am Ende des Projektes produktiv laufen.

2 Theoretische Grundlagen

In diesem Kapitel werden theoretische Grundlagen zu später verwendeten Fachbegriffen erklärt. Dies schließt zunächst den Begriff E-Commerce ein, für den auch eine klare Abgrenzung zum Begriff E-Business getroffen wird. Ebenso wird der Begriff Micro Frontend Frameworks im Zuge dieses Kapitels erläutert, sowie die schaefflereigenen Engineering Apps vorgestellt werden. Diese sind der Anwendungsfall auf den später die gefundene Lösung angewendet wird.

2.1 Definition von E-Commerce Plattformen

Um nicht nur bestehende Kunden bedienen zu können, sondern auch neue Kunden zu gewinnen, sowie den Produktkatalog abzubilden und weiterführende Informationen zu den Produkten aktiv aufzuarbeiten, bietet Schaeffler seine Produkte auch auf E-Commerce-Plattformen an. Diese Plattformen enthalten auch einige von Schaeffler entwickelte Engineering-Apps, wie die Grease App oder den neuen CO2 Calculator, um die Funktionen der Produkte besser für Kunden darstellen zu können.

E-Commerce steht dabei für Electronic Commerce, womit der elektronische Handel von Produkten und Dienstleistungen im Internet bezeichnet wird. Beim E-Commerce können Werbemaßnahmen, Kauf und Verkauf von Produkten in der Regel ohne Zeitverzögerung abgewickelt werden, da alles online geschieht [vgl. 19, Seite 49]. Andere Begriffe die gleichbedeutend mit E-Commerce verwendet werden können, sind beispielsweise „Onlinehandel, Internethandel, Onlineshopping, E-Retailing, Internetvertrieb oder Onlinevertrieb“ [14, Seite 2]. Auf den digitalen Handelsplattformen können sowohl die Waren von Einzelhändlern als auch von Großunternehmen nebeneinander angeboten werden [vgl. 17, Seite 3]. Zu den größten E-Commerce-Plattformen zählen Amazon, Ebay, Wish, Alibaba und Zalando.

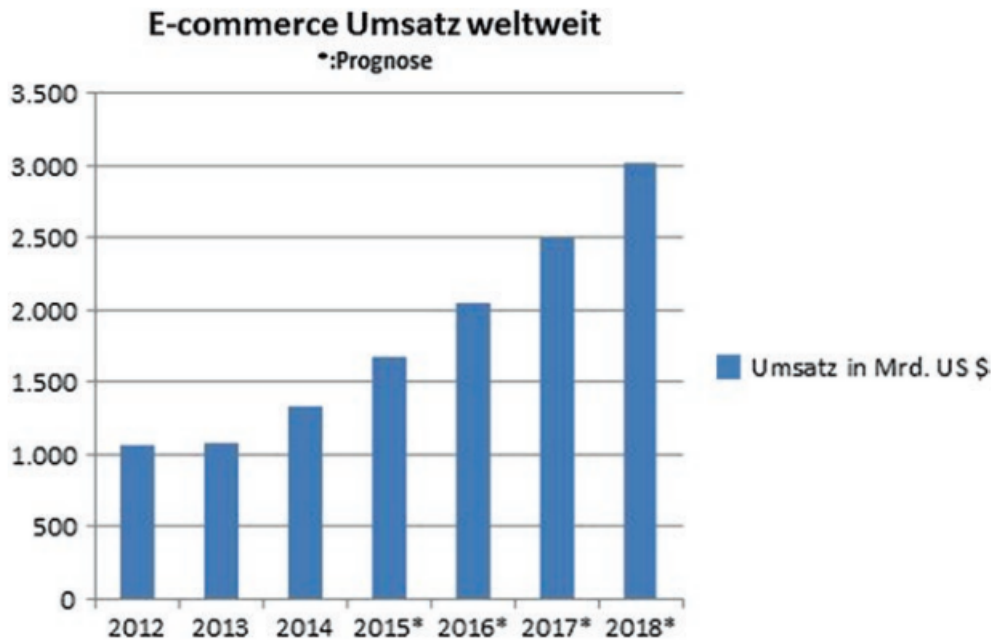


Abbildung 2.1: E-Commerce Umsatz Weltweit (Bildquelle: [17, Seite 4])

Laut einer Studie aus dem Jahr 2018 hat der Onlinehandel ebenfalls in den letzten Jahren stark zugenommen, wie in der vorigen Grafik zu sehen ist. Somit ist davon auszugehen, dass E-Commerce eine entscheidende Rolle im Vertrieb und Einkauf von Waren weltweit spielt. Der Umsatz von Waren über das Internet hat sich dabei weltweit zwischen 2012 und 2018 fast verdreifacht und in Zeiten der globalen Pandemie noch weiter zugenommen.

E-Commerce Plattformen werden jedoch nicht nur im Privatsektor verwendet, sondern auch in der Industrie. Dazu werden branchenspezifische Plattformen verwendet, wie das von Schaeffler genutzte Medias.

E-Commerce kann allerdings nicht mit E-Business gleichgesetzt werden, da E-Commerce lediglich einen Teil des E-Business darstellt. Während E-Commerce die Bezeichnung für die getätigten Transaktionen ist, wird durch den Begriff E-Business auch der gesamte restliche Wertschöpfungsprozess beschrieben, der „auf Basis von computergestützter Internettechnologie“ geschieht [14, Seite 4].

Die folgende Abbildung verdeutlicht nochmals die klare Abgrenzung zwischen E-Commerce und E-Business. Während E-Business neben digitalen Vertriebs- und Transaktionsprozessen auch RFID-Operationen und digitales Supply-Chain-Management umfasst, beschränkt sich E-Commerce ausschließlich auf den digitalen Vertrieb und Transaktionen.

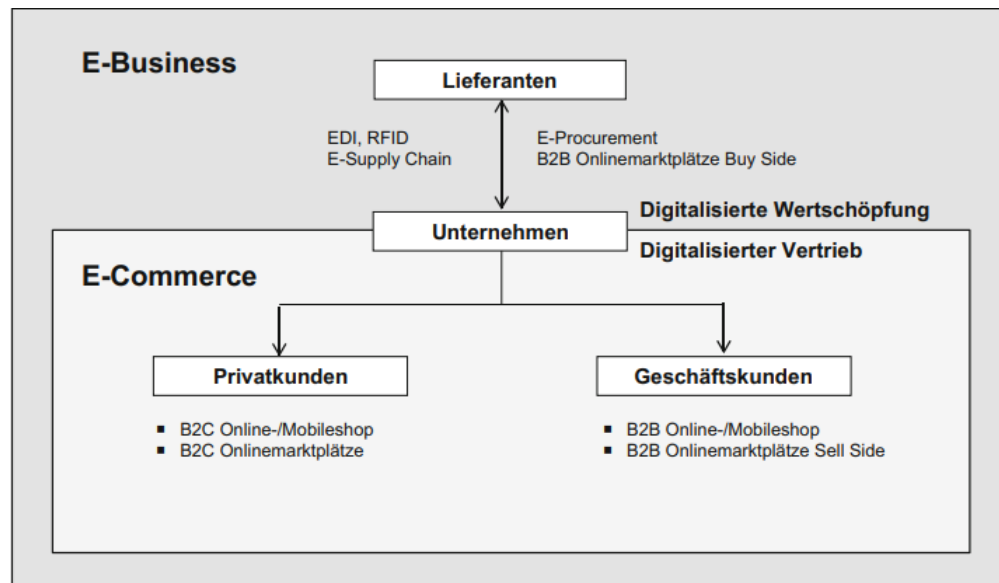


Abbildung 2.2: Abgrenzung zwischen E-Commerce und E-Business (Bildquelle:[14, Seite 5])

Für Käufer bietet der Online-Handel ebenfalls viele Vorteile. E-Commerce ist für alle beteiligten Parteien "günstiger, flexibler, innovativer und dynamischer als die tradierten Formen des Einzel- und Großhandels" [17, Seite 3]. Besonders der Ortsfaktor spielt dabei eine entscheidende Rolle, da Waren beim E-Commerce von überall auf der Welt erworben und anschließend mittels einer Spedition an den Kunden versendet werden können.

Zusätzlich können auf diese Weise wichtige Produktspezifikationen und Informationen vom Kunden zielgerecht und zeitnah eingeholt werden [vgl. 19, Seiten 49 bis 51].

Gleichzeitig verliert man aber auch den direkten Kontakt zum Kunden, was eine Kundenbindung gegebenenfalls erschweren kann.

2.2 Definition von Micro Frontend Frameworks

Durch die technologischen Fortschritte der letzten Jahre werden moderne Webanwendungen immer größer und komplexer. Deshalb werden die verschiedenen Bestandteile der Anwendungen oft von verschiedenen Teams parallel entwickelt [vgl. 10, S. 1]. Um diese Herausforderung zu meistern, setzen immer mehr Unternehmen auf sogenannte Micro Frontend Frameworks. Bei diesen handelt es sich um mehrere kleine Web-Anwendungen, die zu einer größeren Webanwendung zusammen gebaut werden können und welche vom Kunden als eine einzige, zusammengehörige Anwendung wahrgenommen werden [vgl. 9, S. 2]. Die Komponente, in der die anderen Teilanwendungen integriert werden, wird dabei oft als Parent-Komponente bezeichnet. Die zu integrierenden Anwendungen können ebenfalls als Plugins bezeichnet werden. Dabei können diese sowohl mit verschiedenen Backend-Frameworks als

auch in verschiedenen Programmiersprachen entwickelt werden, die über eine Launch App miteinander kommunizieren [vgl. 31].

Der Einsatz von Micro Frontends birgt sowohl Vor- als auch Nachteile.

Ein Vorteil von Micro Frontend Architekturen ist, dass die zu entwickelnden Anwendungen vergleichsweise klein sind und unabhängig voneinander parallel entwickelt werden können. Die Gesamtanwendung kann dabei in beliebig viele, kleine Bestandteile unterteilt werden. Diese Unterteilung geschieht in der Praxis oft entweder nach Features oder nach Seiten. Dadurch werden die einzelnen Komponenten auch für die Entwicklerteams einfacher zu verstehen, da ein Micro Frontend oft von einem einzigen Team entwickelt wird [vgl. 10, S. 1].

Auch die Entwicklung, Testung und das Deployment der einzelnen kleinen Webanwendungen wird durch die Nutzung von Micro Frontends erleichtert. Die Komponenten können einzeln entwickelt werden, was eigene Unit-Tests ermöglicht. Dadurch können die Anwendungen unabhängig voneinander bereitgestellt werden [vgl. 31].

Durch die parallele Entwicklung von Features wird auch die App-Entwicklung insgesamt schneller, da - wie bereits genannt - keine Abhängigkeiten zu anderen Teams entstehen, die Wartezeiten verursachen könnten [vgl. 13].

Auch muss keine Rücksicht auf etwaige veraltete Architekturen genommen werden, wenn eine neue App entwickelt wird. Durch Micro Frontends können verschiedene technische Entwicklungen und Programmiersprachen in den verschiedenen Features verwendet werden, da diese nicht voneinander abhängig sind, aber trotzdem miteinander kombiniert werden können [vgl. 31].

Ein entscheidender Nachteil besteht allerdings darin, dass die verschiedenen Anwendungen aufgrund der parallelen Entwicklung durch unterschiedliche Teams möglicherweise nicht zusammenpassend wirken können. Dies kann daran liegen, dass verschiedene Herangehensweisen bei der Entwicklung verwendet werden, wodurch die Anwendungen am Ende optisch nicht zusammengehörig wirken [vgl. 1].

Ebenso kann die Kommunikation zwischen den verschiedenen Teams durch die Aufteilung der Features beeinträchtigt werden, wenn keine regelmäßigen Absprachen stattfinden. Es kann dann zu Missverständnissen bei der Umsetzung wichtiger Features kommen, die erst beim Deployment der Applikation auffallen [vgl. 1].

Trotz der genannten Herausforderungen ist die Verwendung von Micro Frontends bei großen und komplexen Web-Apps für viele Entwickler sehr attraktiv, weshalb es zahlreiche Möglichkeiten zur Erstellung von Micro Frontends gibt [vgl. 9, S. 2]:

1. Iframes
2. Web Component

3. Angular Libraries

Die Möglichkeiten zur Erstellung von Micro Frontends mittels Iframes und Web Components werden in nachfolgenden Kapiteln genauer betrachtet, da laut Fachabteilung die Implementierung als Angular Library für den Use Case der Engineering Apps nicht funktionieren würde, da Medias kein Angular Framework verwendet. Zusätzlich wurde noch eine sogenannte Script Embed Option erwähnt, die sehr ähnlich wie ein iframe funktioniert, allerdings eher für statische Plugins verwendet wird, weshalb sie nicht bei der Lösungsfindung berücksichtigt wird.

2.3 Überblick über die schaefflereigenen Engineering Apps

Um die Auswahl eines Lösungsmusters besser zu erklären, ist es zu Beginn erforderlich, einen Überblick über die Engineering Apps bei Schaeffler zu geben.

Engineering Apps sind einerseits ein Unterteam des Teams Custom Data and AI 2 bei Schaeffler, andererseits aber auch ein zusammenfassender Begriff für mehrere von Schaeffler entwickelte Anwendungen. Diese entstanden in Kooperation zwischen dem Bearingx-Team von Schaeffler und der bereits erwähnten Produktsuite Custom Data and AI 2, kurz CDAI2, vor zwei Jahren. Das Ziel davon ist es laut Abteilungsbetreuer Robert Krause, das Knowhow und Ingenieurwissen von schaefflereigenen Produkten öffentlich verfügbar zu machen, wofür allerdings das Frontend Know How gefehlt hatte, weshalb das Team Bearingx Unterstützung von Custom Data and AI 2 benötigt hat. CDAI2 kümmert sich dabei um die Darstellung der Apps im Frontend, während Bearingx die technischen Hintergründe und Berechnungsmethoden behandelt.

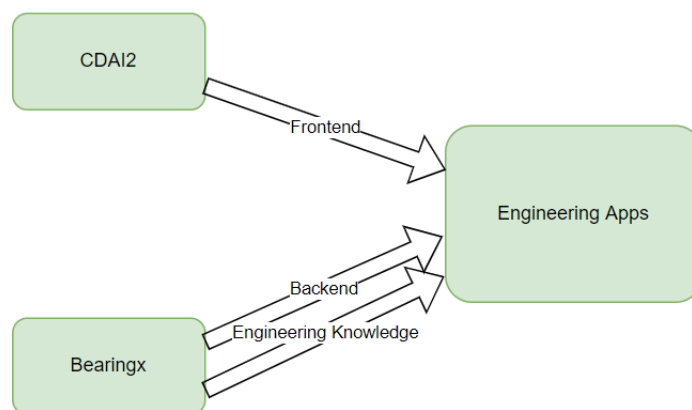


Abbildung 2.3: Kooperation zwischen Bearingx und CDAI2

Dadurch sind die Engineering Apps in gewisser Weise auch ein Reputationsprojekt um das Know How und die Qualität der Schaeffler Produkte aufzuzeigen, und dies als Cloud Lösungen auf den ersten Blick verfügbar zu machen. Dabei ist ebenfalls die Kooperation mit der E-Commerce Plattform Medias zentral, da sie für viele Kunden der erste Berührungspunkt mit den Engineering Apps ist. Das Ziel ist es durch den möglichen Vergleich von Fremdprodukten und Schaefflerprodukten die Kunden zum Kauf von Schaeffler Produkten zu bewegen und somit auch eine Kundenbindung zu generieren (vgl. Anhang Vision der Engineering Apps).

Die Engineering Apps funktionieren dabei sowohl als Einbindung in Medias als auch als Standalone Version. In beiden Versionen liegt der Fokus vor allem auf User Experience und einer möglichst einfachen Nutzung, die aber trotzdem auf Schaeffler Standards angepasst ist. Ebenfalls erlauben es die Engineering Apps User Tracking zu betreiben und somit anonymisiert Informationen über Nutzerzahlen und Nutzerverhalten innerhalb der Anwendung zu erfassen, um die Designs auch möglichst an die Kunden anzupassen. Die Engineering Apps sind dabei in möglichst vielen Sprachen und Einheitssystemen verfügbar, um eine optimale Anpassung an den Nutzer gewährleisten zu können. Trotz dieser Nutzerzentrierungen sind jedoch alle Berechnungen laut den Experten fundiert und basieren auf physikalischen Berechnungsmethoden, die genauso auf dem Papier gerechnet werden könnten. Die Apps vereinfachen diese Berechnungen nur und tragen sie an die Öffentlichkeit, in der Hoffnung durch die Offenlegung des Know Hows höhere Gewinne zu generieren, wie der Abteilungsbetreuer Robert Krause in einem Expertengespräch erzählte.

Für die Engineering Apps gibt es viele verschiedene Nutzerzielgruppen, die die Apps erreichen sollen. Darunter zählen zwar im Allgemeinen auch die Kunden, die die Produkte von Medias kaufen, aber auch die unternehmensinternen Techniker, sowie die externen Instandhaltungsbeauftragten, die beim Kunden die Lager einbauen und pflegen, um eine falsche Nutzung der Schaeffler Produkte zu verhindern (vgl. Anhang Vision der Engineering Apps).

Die erste Engineering App die in dieser Kooperation entstand war der Mounting Manager. Dieser erlaubt es basierend auf dem vom Nutzer ausgewählten Lager den genauen Einbau in eine beliebige technische Anwendung zu spezifizieren, indem einige Rahmenkonditionen eingegeben werden, und am Ende der Berechnung ein Bericht zur korrekten Montage des Lagers ausgegeben wird. Das Ziel des Mounting Managers ist es vor allem defekte Teile zu vermeiden indem Montageparameter angegeben werden und auch die korrekten Montagewerkzeuge angegeben werden. Nach dem Erfolg des Mounting Managers folgte dann laut Robert Krause die Erstellung der Grease App. Diese App zeigt dem Nutzer auf, wie ein Schaeffler Lager nachgefettet werden muss unter der Berücksichtigung der Last, Drehzahl und der anderen Betriebsbedingungen des Lagers.

The screenshot displays the 'Betriebsbedingungen für 6268-M' configuration screen in the Grease App. It is divided into three main sections: 'Lasten' (Loads), 'Bewegungseinflüsse' (Movement influences), and 'Umgebungseinflüsse' (Environmental influences). Each section contains mandatory input fields marked with an asterisk and an information icon. The 'Lasten' section includes a 'Radial- und Axiallast eingeben' toggle and a 'Belastungsannahme' dropdown. The 'Bewegungseinflüsse' section features a 'Art der Bewegung' dropdown set to 'rotierend' and a 'Drehzahl' input set to '1/min'. The 'Umgebungseinflüsse' section has 'Betriebstemperatur' (70 °C) and 'Umgebungstemperatur' (20 °C) inputs, along with an 'Umgebungseinfluss' dropdown set to 'mittel'. A green banner labeled 'NEU: Produktvorauswahl (optional)' is present, containing a 'Fett auswählen' dropdown set to 'Alle Fette' and a 'Kartuscheneinstellung berechnen (CONCEPT1)' toggle. At the bottom, there are buttons for 'EINGABEN ZURÜCKSETZEN' and 'ERGEBNISSE ANZEIGEN'.

Abbildung 2.4: Benutzeroberfläche der Grease App

Dabei wird nach Eingabe der Parameter schließlich angezeigt, welche Menge Fett in welchem Zeitintervall nachgeschmiert werden muss, damit das Lager nicht mit zu wenig Schmierung genutzt wird oder Fett verschwendet wird. Dies soll dazu führen, Lagerschäden zu vermeiden und eine möglichst hohe Lebensdauer des Lagers zu gewährleisten. Zusätzlich hilft die Grease App aber auch einen geeigneten Schmierstoff zu finden, indem ein Vorschlag generiert wird, der Schaefflereigene Fette mit Fremdfetten vergleicht und somit die Nutzer überzeugen soll, die Fette von Schaeffler zu kaufen. Dies war laut Betreuer Robert Krause ein bisher nicht erschlossener Markt für Schaeffler, der zum jetzigen Zeitpunkt schon mehrere Millionen Euro Benefit für Schaeffler generiert hat. Auch ist die Grease App eine Engineering App, für die das Nutzer Tracking schon angewendet wird, indem durch Application Insights in der Standalone Version, und durch Google Analytics bei der Medias Einbindung das Nutzerverhalten erfasst wird.

Eine neu entwickelte Anwendung, die Bestandteil der sogenannten Engineering App ist, ist der sogenannte CO2 Calculator, der sämtliche CO2 Emissionen der Schaeffler Produkte bei Betrieb und Produktion berechnet. Dies soll Klarheit schaffen über die Emmissionsabgaben und auch in Zukunft durch die nachhaltige Verwendung der Komponenten einen Teil des notwendigen Emissionshandels vermeiden.

Die Engineering App ist eine Kombination aus mehreren Berechnungsmethoden und auch zusätzlich zu den technischen Berechnungen zur Lagernutzung den Co2 Calculator beinhaltet.

Engineering App

Operating conditions for bearing 6210-C-2HRS

RESET INPUTS

⚙️ Loads
* required information

Radial load * N ⓘ

Axial load * N ⓘ

🌀 Rotational speed
* required information

Rotational speed * 1/min ⓘ

Choose your calculations

☒ Calculations

☒ CO₂ emissions

☐ Overrolling frequency

BASIC FREQUENCIES

ⓘ Please add the remaining conditions.

🌿 Total value of CO₂:

Production	Operation
1.33 kg	-

SHOW REPORT

Abbildung 2.5: Benutzeroberfläche der Engineering App

Zusätzlich zu dem am unteren Ende angebrachten CO₂ Calculator, verfügt die Engineering App jedoch auch über eine Lebenszeitberechnung, die angibt, wie lange das Lager bei den angegebenen Parametern funktionsfähig ist. Dazu existiert ebenfalls eine Berechnung des Frictional Power Loss, der laut Schaeffler Experten aussagt, wie viel Kraft des Lagers durch Reibung verloren geht, und ein „Lubrication Parameter“ der anzeigt, wann ein Lager nachgeschmiert werden muss. Auch kann man, sofern notwendig bei der Engineering App einen Report aller notwendigen Parameter anzeigen lassen, und diesen schließlich zu einer PDF konvertieren, um ihn zu verschicken. Dies ist letztendlich auch die App, die in Medias mithilfe einer Web Component neu eingebunden werden soll.

3 Vorgefundene Lösungsmuster mit Vergleich

In diesem Kapitel werden die Kriterien, die Medias, Deloitte und Schaeffler zur Einbindung der Engineering App gesetzt haben erläutert. Anschließend werden die Grundlagen zweier vorgefundener Lösungsmuster näher betrachtet. Dabei handelt es sich um den bereits vorherrschenden iFrame und die Nutzung von Web Components. Anschließend werden die Vor- und Nachteile der Lösungsmethoden miteinander verglichen und auf Basis dieser wird eine Entscheidung für eine Methode gefällt. Das hier dargestellte Wissen wurde zum Großteil aus Onlinerecherchen sowie einem Termin mit den Medias Kollegen, Deloitte und Schaeffler Mitarbeitern gewonnen.

3.1 Kriterien zur Auswahl der Lösungsmuster

Damit das gefundene Lösungsmuster auch passend für den Use Case ist, wurden zu Beginn sowohl von Medias als auch seitens Engineering Apps bereits Kriterien festgelegt, die die Lösung erfüllen muss, um von beiden Seiten akzeptiert zu werden.

Das vermutlich wichtigste Kriterium dabei war, dass die Einbindung so einfach wie möglich für die Entwickler des Medias Portals gestaltet werden soll. Die Entwickler, die die Engineering App in Medias letztendlich einbinden, sind Entwickler von Medias externen Partnern Deloitte, während bei Schaeffler und Medias IT-Verantwortliche und Projektmanager zu finden sind. Durch die Kooperation zwischen Medias, Schaeffler und den Entwicklern von Deloitte sind somit insgesamt 3 Parteien an der Medias Einbindung der Apps beteiligt, und jede zusätzliche Abstimmung würde Zeit kosten. Dies bedeutet im weiteren, dass bei Updates kein zusätzlicher Aufwand im Sinne von Hotfixes oder manuellen Updates bei Medias oder Deloitte entstehen soll. Im schlimmsten Fall könnten hier, wenn Updates manuell ausgeführt werden müssen, verschiedene Stände zwischen der Standalone Version und Medias entstehen, sodass die Medias Einbindung veraltet wäre, sofern diese die Updates nicht sofort übernehmen. Bei einer Eingabe von falschen Werten in die Anwendung wäre dies verheerend, da die Berechnungen falsche Werte liefern würden, die andernfalls vielleicht in der neuen Version nicht mehr zugelassen wären.

Des weiteren ist allen Parteien wichtig, dass mit der Einbindung durch eine Web Component kein technischer Rückschritt entsteht. Diese Gefahr besteht in dem Fall, dass bereits bestehende Technologien wie Tracking durch die neue Einbindung außer Kraft gesetzt werden.

Dies soll verhindert werden. Bereits existierende Features sollen auch nach der neuen Einbindung entweder direkt weiter funktionieren oder Konzepte für die zukünftige Entwicklung der Features müssen vorhanden sein. Vor allem das User Tracking spielt dabei eine wichtige Rolle, da auch Schaeffler daran interessiert ist, wie die bereitgestellten Anwendungen genutzt werden.

Zudem soll - wenn möglich - eine Variante gewählt werden, bei der der Inhalt der in der Webcomponent eingebundenen Seite auch von Suchmaschinen gesehen werden kann, wodurch sich Schaeffler mehr Zugriffe auf der Seite verspricht.

Dazu kommt letztendlich aber auch das optische Design der Seite, die zum jetzigen Stand optisch unoptimiert aussieht. Dies soll durch eine neue Einbindung vermieden werden, sodass die Einbindung optisch stimmig wird und auch responsive ist. Dennoch sollen die Styles der Medias Seite und die der Anbindung sich nicht gegenseitig beeinflussen, da Schaeffler sich an den eigenen Styleguide und dessen Richtlinien gehalten hat. Laut Aussage von Firmenbetreuer Robert Krause kann jedoch nicht gewährleistet werden, dass der Style Guide in beiden Anwendungen immer gleichermaßen umgesetzt ist. An dieser Stelle ist eine Style-Trennung gewünscht, obwohl bewusst ist, dass dies auch eine Verwendung des Darkmodes oder ähnlicher Mechanismen blockieren würde.

Ebenfalls sollte die Lösung in Angular umsetzbar sein, da Angular das zur Erstellung der Engineering Apps gewählte Programmierframework ist. Dieses basiert dabei vor allem auf Typescript und wird oft genutzt um Webanwendungen bei Schaeffler zu bauen. Medias hingegen verwendet kein Angular, was bedeutet dass die in Angular entwickelte Lösung jedoch in ein anderes Framework integriert werden können soll, da die Frameworks auch Daten austauschen sollen, z.B. für ein User Tracking oder um die Funktionalität der Engineering Apps gewährleisten zu können, für die von Medias Daten an die integrierte App übergeben werden müssen.

Zusätzlich soll die gefundene Lösung zudem mit allen gängigen Browsern kompatibel sein, da die Medias Seite ebenfalls von allen Browsern nutzbar ist, und man hier somit Fehler eliminieren möchte, die wiederum zu unoptimiertem Design führen könnten.

3.2 Grundlagen von iFrames

Eine Möglichkeit, externe Applikationen in eine andere Website zu integrieren, ist die Verwendung von Inline-Frames, kurz iFrames. iFrames werden in der Regel verwendet, um ein HTML-Dokument in ein anderes HTML-Dokument einzubetten. Dafür wird im HTML-Code der Website in die eingebettet werden soll, ein `<iframe>`-Tag verwendet, gefolgt von der Quelle, an der der eingebettete Code liegt, sowie weiteren Spezifikationen wie Styles oder ähnliches [vgl. 25].

Das kann folgendermaßen aussehen:

```
1 <body>
2     <iframe src="https://fr.wikipedia.org/wiki/Main_Page"
3         style="border: 3">
4     </iframe>
5 </body>
```

Listing 3.1: Einbindung einer Anwendung mittels iFrame (Codequelle: [8])

Der iFrame-Tag wird, wie im obigen Code beschrieben, im Body des HTML-Dokuments der übergeordneten Seite platziert, während die Quelle angibt, wo sich der Code des eingebetteten Features befindet.

So „entsteht ein verschachtelter Browsing-Kontext, in den nützliche Inhalte von Websites oder Diensten von Drittanbietern ohne großen Konfigurationsaufwand direkt eingefügt werden können“ [vgl. 11].

In der Praxis gibt es viele Anwendungsfälle, bei denen iFrames genutzt werden können, um Content von einem Drittanbieter auf der eigenen Seite anzuzeigen. Dazu zählen unter anderem Pop-Up Werbung, YouTube-Videos, die Einbindung von Google Maps, Social-Media-Posts und Newslettern. Auch Single-Sign-On-Services, Echtzeit-Widgets, Texteditoren und zahlreiche andere Web-Apps können über iFrames eingebettet werden [vgl. 22].

Auch die Nutzung von iFrames bietet Vor- und Nachteile.

Einer der Vorteile von iFrames besteht darin, dass der inline Frame komplett von der Website isoliert ist. Deshalb haben Änderungen innerhalb des iFrames keinen Einfluss auf die übergeordnete Website. Zudem sind die Betreiber der übergeordneten Website nicht für Updates des im iFrame angezeigten Contents verantwortlich, da dieser sich in vielen Fällen selbst aktualisieren kann [vgl. 22].

Darüber hinaus sorgt diese Isolierung auch für eine erhöhte Sicherheit der Website, da das Risiko vermieden wird, gemeinsam mit dem Code eines Drittanbieters etwaige Angriffspunkte in den Code der eigenen Website zu importieren [vgl. 22].

Ein weiterer Vorteil von iFrames ist die einfache Handhabung bei der Implementierung, da sie - wie bereits erwähnt - fast nur aus HTML-Elementen bestehen und somit einfach zu implementieren sind [vgl. 15].

Eine Einbindung mittels iFrame birgt jedoch auch gewisse Nachteile.

Einesseits kann der iFrame auch als Sicherheitsrisiko angesehen werden, da jeglicher Content, der im iFrame angezeigt wird, auch auf der Seite angezeigt wird. Dadurch kann das iFrame zur Zielscheibe für Hacker werden, die versuchen könnten, Nutzerinformationen durch Phishing oder Cyber Security Lücken abzugreifen. Andererseits kann die Verwendung von

iFrames aber auch die Ladezeit der Website verlangsamen und zu Problemen bei der Suchmaschinenoptimierung führen, da der Content innerhalb des iFrames möglicherweise nicht von Suchmaschinen indiziert wird [vgl. 22, 11].

Die Isolation der Parent-Komponente und der Child-Komponente bei der Nutzung von iFrames hat jedoch auch den Nachteil, dass kein Datenaustausch zwischen den beiden Komponenten möglich ist. Das kann potenzielle Einsätze von Web-Apps zum User-Tracking, wie etwa Google Analytics, nahezu unmöglich machen [vgl. 15].

Außerdem sind iFrames nicht nativ responsiv, was bedeutet, dass sie von einigen Browsern möglicherweise nicht optisch ansprechend dargestellt werden [vgl. 8].

3.3 Grundlagen von Web Components

Eine weitere Option, externe Web Anwendungen in eine bestehende Website zu integrieren, ist die Verwendung von sogenannten Web Components. Statt der normalen HTML-Tags wie `<div>` und `<video>`, werden bei der Erstellung von Web Components mithilfe von TypeScript neue custom HTML-Tags erstellt, die auch in anderen Projekten wiederverwendet werden können [vgl. 29].

Dies geschieht durch eine bereits in Angular integrierte Methode zur Erstellung von Custom-Elements, die ausschließlich auf HTML, CSS und TypeScript basiert. Zusätzlich wird eine sogenannte Shadow DOM verwendet, um Teile der Web Components getrennt von der Main-DOM zu rendern. Dadurch können die Features der Web Component gestaltet werden, ohne dass die Web Component mit anderen Teilen der Website kollidiert. Diese Technik wird auch als Encapsulation bezeichnet [29].

Ein Beispiel für die Implementierung einer Web Component könnte wie folgt aussehen:

```
1 class SpanWithText extends HTMLSpanElement {
2     constructor() {
3         super();
4         this.innerText = 'Hello, I am a Web Component
5         !';
6     }
7 }
8 customElements.define('span-with-text', SpanWithText);
```

Listing 3.2: Einbindung einer Anwendung mittels Web Component (Bildquelle: [29])

Zunächst wird eine Klasse definiert, die beim Aufruf der Klasse den Text "Hello, I am a Web Component" ausgibt. Anschließend wird der entsprechende HTML-Tag generiert, so dass bei Verwendung dieses Tags der Text aus der generierten Klasse angezeigt wird. Dadurch ist es relativ einfach, externe Medien in eine Website zu integrieren, indem anstelle des

Textes auch die Quelle einer anderen Website stehen kann, die dann innerhalb der Web-Components angezeigt wird [vgl. 29].

Auch dieser Ansatz hat Vor- und Nachteile.

Ein Vorteil der Nutzung von Web-Components besteht darin, dass sie mit allen gängigen Webdesign Frameworks kompatibel sind, und somit flexibel mit React, Vue und Angular verwendet werden können [vgl. 26].

Die generierten HTML-Tags können auch in zukünftigen Projekten einfach wiederverwendet werden und sind somit recycelbar. Außerdem haben sie in der Regel nativen Browser-Support, da sie nur aus HTML-, CSS- und JavaScript-Komponenten bestehen, was zu einer responsiven Website führt. Ein weiterer Vorteil ist, dass zur Erstellung von Web Components in der Regel keine komplexen Abhängigkeiten installiert werden müssen [vgl. 26].

Ein weiterer Vorteil der Web Components besteht darin, dass durch die Verwendung von sogenannten Input-Parametern, die in der Web Component spezifiziert werden, Daten von der übergeordneten Component an die Web Component übergeben werden können. Dadurch ist es möglich, Funktionen wie das Tracking per Google Analytics zu implementieren [vgl. 6].

Eine Web Component hat jedoch auch Nachteile, wie zum Beispiel, dass die Website möglicherweise länger zum Laden benötigt, da die Web Component separat geladen werden muss [vgl. 26].

Außerdem können durch die bereits erwähnte Encapsulation keine Styles zwischen der übergeordneten Komponente und der Web Component geteilt werden, was dazu führen kann, dass die Elemente möglicherweise optisch nicht zusammenpassen [vgl. 26].

3.4 Überblick und Vergleich der gefundenen Lösungsmuster anhand der von Engineering Apps und Medias gesetzten Bedingungen

Um im weiteren Vorgehen eine angemessenes Lösungsmuster bestimmen zu können, werden an dieser Stelle die bereits in Abstimmung mit Schaeffler und Medias bestimmten Kriterien herangezogen. Diese werden von beiden Seiten als besonders wichtig erachtet und bestimmen maßgeblich über eine geeignete Lösung, da dies Kriterien sind, die den aus dem Use Case extrahiert wurden.

Die vorgefundene Lösungen werden anhand dieser Kriterien überprüft. Dabei werden insbesondere die Vor- und Nachteile der gefundenen Lösungen berücksichtigt.

iFrames	Web Component
komplett von der Parent Website isoliert	Frameworkunabhängig mit nativem Browsersupport
höhere Sicherheit	Komponenten recyclebar
einfache Handhabung da nur HTML Komponenten	keine komplexen Dependencies
	Datenaustausch mit Parent Website

Tabelle 3.1: Vorteile der Lösungsansätze

Nach erster Literaturrecherche ist festzustellen, dass, wie in Tabelle 3.1 zu sehen ist, im Falle der Web Component ein Datenaustausch zwischen der Parent Website - in diesem Fall Medias - und der eingebundenen Seite möglich ist. Dieser funktioniert mittels input Parametern, welche in der Web Component generiert werden und schließlich in der Parent Seite genutzt werden können, um Beispielsweise Lagerbezeichnungen übergeben zu können, wodurch die App lauffähig bleibt. Bei einer Einbindung mittels iframe ist dies nur bedingt möglich, da zwar Lagerbezeichnungen übergeben werden können, jedoch weitere Use Cases wie User Tracking erschwert werden, während diese bei einer Web Component leichter umsetzbar sind. Das iframe profitiert dabei jedoch von einer höheren Sicherheit, da es von außen nur schwer beeinflusst werden kann und somit ebenfalls nur schwer von außen manipuliert werden kann.

Ein Vorteil beider Optionen ist jedoch die relativ einfach gehaltene Umsetzung, die keine komplexen Abhängigkeiten zu den anderen Frameworks erfordert, da beide Frameworks hauptsächlich auf HTML Komponenten basieren, was darauf schließen lässt, dass hinsichtlich externer Abhängigkeiten, die installiert werden müssen, kein großer Aufwand auf Medias und Schaeffler zukommen sollte. Zudem sind die Lösungsmuster unabhängig vom gewählten Framework nutzbar und somit mit allen Browsern kompatibel.

Die Lösungsmuster verfügen aber auch über entscheidende Nachteile.

Ein bedeutender Nachteil von iFrames ist jedoch, dass diese nicht nativ responsive sind und somit auf manchen Geräten nicht richtig angezeigt werden können. Das zeigt sich durch optische Verschiebungen im Style der Website.

iFrame	Web Component
Sicherheitsrisiko	längere Ladezeiten
kein Datenaustausch möglich	kein Stylesharing wenn Encapsulation verwendet
langsamere Website	Veränderung der Web Component durch Input Parameters
nicht nativ responsive	

Tabelle 3.2: Nachteile der Lösungsansätze

Darüber hinaus gibt es Sicherheitslücken bei beiden Methoden, die jedoch bei Verwendung von Web Components durch Encapsulation behoben werden können. In diesem Fall ist jedoch kein Stylesharing mehr zwischen der Web Component und der Parent-Website möglich, was bedeutet dass die Seiten absolut voneinander optisch isoliert sind.

Zudem sind diese beiden Methoden langsamer beim Laden der Seite, als wenn keine Einbindung stattfinden würde.

Aufgrund der genannten Vorteile der Web Component, sowie des ausdrücklichen Wunsches der Medias Entwickler, wurde sich dazu entschieden den Einsatz einer Web Component zur Einbindung der Engineering Apps in Medias zu testen.

Diese Entscheidung wurde gefällt, obwohl alle Theoretischen Materialien, die zu dem Thema gefunden wurden nur für rudimentäre Anwendungen nutzbar sind, und die Dokumentationen nicht alle bei der Engineering App benötigten Use Cases abdecken.

Dennoch war diese Lösungsmethode explizit von sowohl der Abteilung als auch Medias gewünscht, weshalb im folgenden eine Web Component in einem Praxisbeispiel getestet wird.

4 Ansatz zur Nutzung der Web Component

In diesem Kapitel wird die Web Component in einem Praxistest auf gewünschte Kriterien geprüft. Dazu zählen Tests zur Browserkompatibilität und zur Werteübergabe, da diese Features maßgeblich für die Eignung der Web Component sind. Daraufhin werden zwei Herausforderungen bei der Implementierung einer Web Component identifiziert, für welche anschließend auch Lösungsansätze erarbeitet werden. Im letzten Teil des Kapitels wird schließlich ein grobes Konzept erarbeitet, wie das Vorgehen bei der Implementierung der Engineering App als Web Component sein muss. Das notwendige Wissen zur Bearbeitung wurde vor allem durch Lesen der Angular Dokumentationen und Mitarbeit in der Abteilung erlangt.

4.1 Praxistest der Web Component als Lösungsmuster

Um die Praxistauglichkeit einer Webkomponente zu evaluieren, muss ein Testansatz umgesetzt werden, der aus verschiedenen Schritten besteht.

Zuerst wird dazu eine auf ihre Hauptbestandteile reduzierte Webkomponente entwickelt, um die optimale Implementierungsstrategie zu ermitteln. Anschließend müssen Methoden zur Übermittlung von Eingabeparametern getestet werden, um den Datenaustausch zwischen der Medias Seite und der Engineering App zu gewährleisten. Um sicherzustellen, dass die Webkomponente von verschiedenen Browsern einheitlich dargestellt wird, wird ein Test mit verschiedenen gängigen Browsern durchgeführt. Darüber hinaus soll geprüft werden, ob die Webkomponente auf einem lokalen Server ausgeführt werden kann, da die Engineering-Apps unabhängig von anderen Websites auch als Standalone Version lauffähig sein sollen.

Um eine Web Component für Testzwecke implementieren zu können, wird sich an einem Guide von Ahmed Boucheфра orientiert, der sich zu Beginn als guter Leitfaden erweist, um die Angular Installationen und die wichtigsten Parameter festzulegen. In Folgeschritten ist jedoch festzustellen, dass der Leitfaden veraltet ist, da einige Befehle nicht mehr existieren. Dennoch bietet er für die erste Implementierung einer Web Component einen guten ersten Überblick.

Damit mit der Implementierung gestartet werden kann, muss zuerst Angular installiert werden, wofür eine Installation von node.js und npm notwendig ist. Diese Abhängigkeiten

müssen heruntergeladen werden, bevor schließlich die Angular CLI installiert werden konnte, mit der man eigene Angular Projekte erstellen kann. Die Installation wird mit folgendem Angular Befehl getätigt:

```
1 npm install -g @angular/cli
```

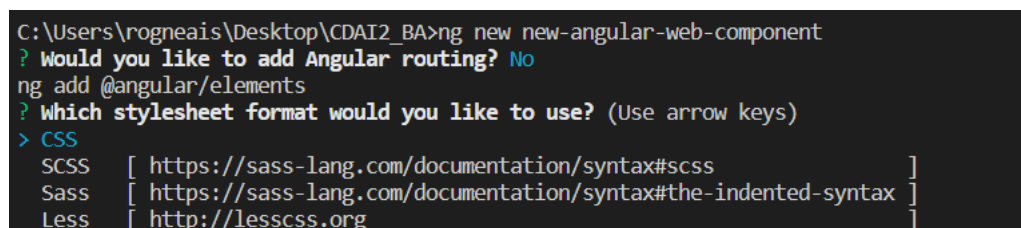
Listing 4.1: Angular Installation

Anschließend wird ein neues Angular-Projekt in einem frei wählbaren Verzeichnis erstellt. Für die Testanwendung wird das Verzeichnis "new-angular-web-component" gewählt. Hierfür wird ein Befehl in der Kommandozeile des Laptops verwendet, gefolgt von einem Wechsel in das neu erstellte Verzeichnis. Im Anschluss wird das Angular Elements-Modul in dieses Verzeichnis importiert, wie im folgenden Code zu sehen ist:

```
1 ng new new-angular-web-component
2 cd new-angular-web-component
3 ng add @angular/elements
```

Listing 4.2: Initialisierung eines Angular Projekts

Während der Ausführung des vorigen Codes aus Listing 4.2, werden zwei Abfragen getätigt. Sowohl der vorgeschlagene Parameter für das Routing als auch das Format des Stylesheets können dabei vom Nutzer gewählt werden. In diesem Testbeispiel wurde sich für eine App mit CSS-Stylesheet entschieden, da dies der Engineering App am ähnlichsten erschien.



```
C:\Users\rogneais\Desktop\CDAI2_BA>ng new new-angular-web-component
? Would you like to add Angular routing? No
ng add @angular/elements
? Which stylesheet format would you like to use? (Use arrow keys)
> CSS
SCSS [ https://sass-lang.com/documentation/syntax#scss ]
Sass  [ https://sass-lang.com/documentation/syntax#the-indented-syntax ]
Less  [ http://lesscss.org ]
```

Abbildung 4.1: Initialisierung eines Angular Projektes

Da Angular Elements allerdings in den ersten Versuchen nicht durch den Befehl hinzugefügt werden konnte, wurde schließlich eine Installation per npm Befehl verwendet, woraufhin das Modul einsatzbereit war und woraufhin auch andere Module automatisch installiert wurden.

Wenn das Setup schließlich bereit ist, hat sich ein leeres Angular Projekt mit einigen Musterdateien gebildet, die in den weiteren Schritten zu einer Web Component umgebaut werden

können. Im folgenden muss schließlich zusätzlich die sogenannte „config“ Datei des Projektes angepasst werden, um zukünftige Fehler bei der Erstellung von HTML Elementen zu vermeiden. Dazu ist es notwendig in der Datei „tsconfig.json“ der „target“ Parameter auf den Wert „es2015“ zu setzen, bevor mit der eigentlichen Arbeit an der Web Component angefangen werden kann. Die Datei befindet sich nahe dem unteren Ende des Ordners [vgl. 12].

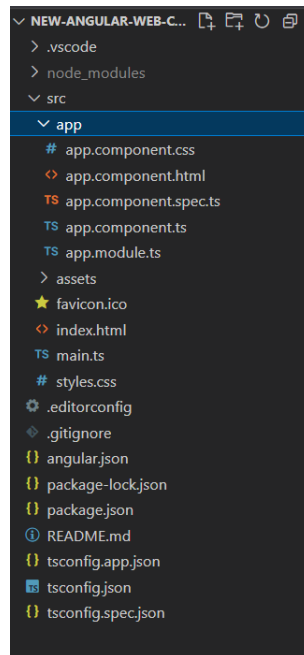


Abbildung 4.2: Ordnerstruktur einer Angular App

Daraufhin kann die Datei „app.module.ts“ geöffnet werden, die bereits Mustercode enthält, welcher zur Erstellung einer Web Component ergänzt werden muss. Zu Beginn müssen zwei zusätzliche Module importiert werden, eines davon „createcustomElement“, was maßgeblich zur Erstellung von Web Components ist. Dieses wird schließlich auch in die Exportklasse der Datei eingesetzt, wo auch der Name der Web Component in einem Constructor festgelegt wird, gefolgt von „Bootstrapping“.

Im Test Beispiel ist der Name des Custom HTML Tag der Web Component „web-app“, da dies kein Name eines bereits existierenden HTML Tags ist und für den Anwendungsfall, wie in Listing 4.3 zu sehen ist, passt [vgl. 12]. Darauffolgend kann dann ein erster Build der App gestartet werden. Dies erfolgt durch den Command „ng build - - output-hashing none“ in der Konsole [vgl. 12]. In diesem Fall wird der Parameter „Output Hashing“ bewusst ausgeschaltet, da sonst im Zuge der Testanwendung für jeden neuen Build auch neue Dateien in die Test HTML Datei eingebunden werden müssten. Da im Zuge der Testung der Web Component allerdings oft neue Builds generiert werden müssen, um verschiedene Inhaltsparameter zu testen, wurde sich entschieden diese Builds immer mit dem Parameter „- - output-hashing none“ zu erstellen, der an den Build Befehl angehängt wird.

```

1 import { Injector } from '@angular/core';
2 import { createCustomElement } from '@angular/elements';
3 import { NgModule } from '@angular/core';
4 import { BrowserModule } from '@angular/platform-browser';
5 import { AppComponent } from './app.component';
6 @NgModule({
7     declarations: [    AppComponent    ],
8     imports: [        BrowserModule    ],
9     providers: [],
10 })
11 export class AppModule {
12     constructor(private injector: Injector){
13         const el = createCustomElement(AppComponent, {
14             injector });
15         customElements.define('web-app', el);
16     }
17     ngDoBootstrap() {}
18 }

```

Listing 4.3: Initialisierung der Test Web Component

Um zu überprüfen, ob auch etwas in der Web Component angezeigt wird, kann dabei ein Web Server verwendet werden, um diese lokal anzeigen zu lassen. Dies geschieht, indem man den Dateien aus dem „dist“ Ordner einen sogenannten „Localhost“ zuweist, über welchen diese anschließend einsehbar sind. Dazu ist der folgende Code in der Konsole vor dem Build der App notwendig:

```

1 npm install http-server -g
2 http-server dist

```

Listing 4.4: Initialisierung eines Web Servers (Codequelle: [20])

Anschließend kann der Inhalt der Web Component im Browser eingesehen werden.

Um die Web Component allerdings auch richtig nutzen zu können, muss diese in eine andere Website eingebunden werden. Für dieses Testbeispiel initialisiert man eine nur aus Basiselementen bestehende HTML Seite, welche im folgenden Anwendungsfall Test genannt wird.

Die Test HTML Seite besteht aus einem Head und einem Body. Im Head wird dabei der Titel der App festgelegt, in diesem Fall Engineering App, während im Body die eigentliche Einbindung der Web Component geschieht. Die Einbindung selbst geschieht dadurch, dass der neu erstellte HTML Tag, der in der Web Component erstellt und gebaut wurde, an diese Stelle eingefügt wird. Innerhalb des Tags können auch andere Werte übergeben werden, die für die Web Component von Bedeutung sind. Daraufhin wird auf die gebauten Scripts der Web Component verwiesen, welche im sogenannten „dist“ Ordner zu finden sind. Für

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width,
        initial-scale=1.0">
6     <title>Engineering Web Apps</title>
7     <base href="/">
8 </head>
9 <body>
10    <p>Das ist die Parent Seite</p>
11    <web-app></web-app>
12    <script type="text/javascript"
13        src="C:\Users\rogneais\Desktop\CDAI2_BA\new-
        angular-web-component\dist\main.js">
14    </script>
15    <script type="text/javascript"
16        src="C:\Users\rogneais\Desktop\CDAI2_BA\new-
        angular-web-component\dist\polyfills.js">
17    </script>
18    <script type="text/javascript"
19        src="C:\Users\rogneais\Desktop\CDAI2_BA\new-
        angular-web-component\dist\runtime.js">
20    </script>
21    <p>Das ist die Parent Seite</p>
22 </body>
23 </html>

```

Listing 4.5: Einbindung der Web Component in eine HTML Seite

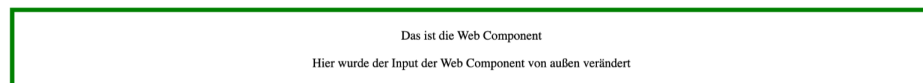
eine rudimentäre Web Component, wie die hier vorliegende, sind nur die Scripts „main.js“, „polyfills.js“ und „runtime.js“ von Bedeutung, für größere Projekte wie die Engineering Apps werden allerdings auch andere Scripts benötigt.

Sobald dies erledigt ist, kann die Test HTML Datei im Browser geöffnet werden.

Das Ergebnis sieht dann aus wie im folgenden Bild zu sehen ist:

Engineering Apps

Das ist die Parent Seite



Wieder die Parent Seite

Abbildung 4.3: Test Web Component in der Browseransicht

Wie in diesem Beispiel bereits zu sehen ist, kann man den Inhalt der Web Component auch von außen verändert werden oder Werte übergeben werden. Das ist vor allem wichtig in

Bezug auf die Engineering Apps, da für diese von Medias Parameter übergeben müssen, die zur Berechnung verwendet werden. Diese Werteübergabe geschieht, indem innerhalb der Web Component sogenannte „Input Parameter“ definiert werden, die später in der HTML Seite genutzt werden können [vgl. 6]. In diesem Testbeispiel, wurde in der Datei „app.component.ts“ zuerst das Modul Input aus dem Angularpaket angular core importiert, bevor in der Export Klasse der Datei der Input Parameter folgendermaßen initialisiert wird:

```
1 @Input() headline_input="Das ist eine Web Component Ausgabe die  
   verändert werden kann";
```

Listing 4.6: Verwendung des Parameters „Headline Input“

Um diesen Input zur Werteübergabe nutzen zu können, wird anschließend ein weiterer Codeabschnitt in der Datei „app.component.html“ an der Stelle eingefügt, an der später der Input Parameter übergeben werden soll. In diesem Testbeispiel ist dies an Stelle eines normalen Texts, an der anstatt des Textes, der Code „<p>{{headline_input}}</p>“ verwendet wird.

Anschließend kann dieser Parameter auch in der HTML Datei verwendet werden, indem innerhalb des Anfangstags der Web Component der Code „headline_input=„Hier wurde der Input der Web Component von außen verändert““ verwendet wird [vgl. 6].

Das Ergebnis ist, wie oben zu sehen ist, dass der ursprüngliche Text der Web Component durch den Input Parameter ersetzt wird, woraus zu schließen ist, dass eine Werteübergabe zwischen Parent Komponente und Web Component gewährleistet ist.

Im Folgenden soll die Web Component in mehreren Browsern überprüft werden, um zu sehen, ob diese tatsächlich in jedem Browser verwendbar sind. Für Testzwecke wurden in diesem Fall Microsoft Edge, Google Chrome, Safari und Firefox getestet. Das Ergebnis dieses Tests war schließlich, dass alle Browser die Web Component korrekt anzeigen, weshalb im weiteren Vorgehen davon ausgegangen wird, dass die Web Component mit allen gängigen Browsern kompatibel ist.

Damit sind die zu Beginn genannten Kriterien erfüllt, und es kann mit der Planung der eigentlichen Umsetzung anhand des Beispiels der Engineering App begonnen werden.

4.2 Identifikation von Herausforderungen bei der Implementierung

Um möglichen Schwierigkeiten bei der Einbindung vorzubeugen, beschloss das Team CDAI2 anhand der vorangehenden praktischen Testung verschiedene Schwierigkeiten zur Einbin-

derung zu überprüfen und im Voraus bereits ein geeignetes Vorgehen für diese zu evaluieren, um mögliche Herausforderungen bei der Implementierung im Voraus zu vermeiden.

Dabei wurden versehentlich bereits zwei Herausforderungen zur Implementierung einer Web Component bei der praktischen Testung identifiziert.

4.2.1 Datei-Caching der benötigten Skripte

Eine Herausforderung, die bei der Implementierung aufkommen kann, sind sogenannte „Hashes“, die an den Namen der gebauten Dateien angehängt und bei der Ausführung des Befehls „ng build“ erstellt werden. Dieses Hindernis fiel erstmals dadurch auf, dass bei der Ausführung des Befehls nur „ng build“ verwendet wurde, und nicht der Zusatz „- - output-hashing none“, der die Erstellung dieser Hashes verhindert.

Durch die Nicht-Verwendung der Hashes, können jedoch auch Probleme bei einer produktiv laufenden App auftreten, weshalb an dieser Stelle eine Herausforderung aufgetreten ist, für die vor der Einbindung der Engineering App mittels Web Component in Medias eine Lösung gefunden werden musste.

Um den Nutzen und die Funktion der Hashes zeigen zu können, wird allerdings zuerst definiert, was Caching ist, da die beiden Begriffe „Hashing“ und „Caching“ eng miteinander verknüpft sind und oft in Kombination auftreten.

Caching im Zusammenhang mit Web Development ist eine Technik, die genutzt wird um die Leistung eines Computer Systems zu verbessern, indem häufig frequentierte Daten in einem sogenannten Cache gespeichert werden. Wenn das System ein Programm häufiger laden muss, wird dieses im Cache gespeichert, sodass dieses Programm schnell, ohne komplett neu abgerufen zu werden, geladen werden kann [vgl. 16]. Auch Webbrowser und Websites verwenden diese Technik, indem der Browser einige Seiten, die aufgerufen werden, im Cache speichert. Wenn man die Seite erneut aufruft, wird zuerst der Cache überprüft, ob in diesem bereits eine Version der Seite zu finden ist, ohne dass diese neu aufgerufen werden muss [vgl. 2]. Caching ist dabei ein grundlegendes Design Feature des HTTP Protokolls, das von den Browsern genutzt wird. Es hat auch viele Vorteile, wie die Tatsache, dass die Netzwerkkosten einer Anwendung stark reduziert werden können, wenn Caching verwendet wird, da die Websites nicht bei jedem Aufruf neu geladen werden müssen. Im Zuge dessen verbessert sich auch die „Responsiveness“ der Anwendung, da geringere Ladezeiten auftreten. Zudem kann Caching auch dazu führen, dass Inhalte während Server gewartet werden immernoch verfügbar ist, da die Seite nicht neu geladen wird, sondern nur aus dem Cache geladen wird [vgl. 16].

Grundsätzlich ist Caching also eine gute Eigenschaft, die jedoch auch einen entscheidenden Nachteil mit sich bringen kann: Neue Updates der gecachten Internetseite können falsch

angezeigt werden, wenn die Website gecached wird. Dadurch könnten besonders im Fall der in Medias eingebundenen Engineering App, Updates nicht beim Nutzer ankommen, da diese immernoch die gecachte Version der Seite verwenden. Ebenso könnten auch zwischen der selbst veröffentlichten Engineering App und der in Medias eingebundenen Version unterschiedliche Stände angezeigt werden, was es laut den Experten aus dem Team Custom Data and AI 2 auf jeden Fall zu verhindern gilt.

Um das Cachingverhalten des Browsers zu beeinflussen, können wiederum Hashes verwendet werden. Diese funktionieren, indem bei jeder neuen Erstellung der App sogenannte Hashes an alle Dateien im „dist“ Ordner, der die fertigen Dateien beinhaltet, angehängt werden. Hashes sind dabei einzigartige Identifizierer, die an Dateien angehängt werden, um eine Versionierung sicher zu stellen. Dateien heißen dann beispielsweise: „main.e8475920r475.js“. Die Zahlen- und Buchstabenfolge, die vor dem Dateitypen steht ist dabei der Hash, der garantiert, dass die Version der App eindeutig identifizierbar ist, da sich bei jedem neuen Build der App auch die Hashes ändern [vgl. 3].

Im Falle der Engineering Apps stellt jedoch auch dies ein Problem dar. Dadurch dass die Hashes sich nach jedem Update der App ändern, müssen somit auch die Dateien, die in Medias zur Einbindung der App hinterlegt sind, geändert werden. Dies stellt vor allem deshalb ein Problem dar, da Medias einen Veröffentlichungszyklus von sechs Wochen hat, während die Engineering Apps einen Veröffentlichungszyklus von zwei Wochen besitzen, was dazu führen könnte, dass - falls die Dateien nicht sofort nach der Schaeffler Veröffentlichung in Medias angepasst werden - zwei parallele Versionen der Engineering App mit unterschiedlichen Entwicklungsständen zeitgleich existieren könnten, was es laut Engineering Apps Team ebenfalls zu verhindern gilt. Die Dateien des alten Entwicklungsstandes würden in diesem Fall nicht mehr existieren und somit auch die Engineering App nicht funktionieren.

Da beide Optionen also keinesfalls angewendet werden sollen, wurde nach einer passenden Alternative gesucht, die gewährleistet, dass die Dateien zwar nicht ausgetauscht werden müssen, aber auch nicht gecached werden.

Dazu wurden zuerst die sogenannten „Output Hashing“ Optionen überprüft, die bereits in Angular integriert sind, um zu prüfen ob es eine Möglichkeit gibt, die Hashes optional zu setzen und somit - sofern der Hash sich verändert hat - die neue Version zu laden, ohne den Dateinamen ändern zu müssen.

Bei Angular existieren vier Varianten, Output Hashing zu betreiben - oder eben nicht, die in der Angular Dokumentation spezifiziert sind.

Die Variante „ - - output-hashing none“ wurde bereits getestet und ist aufgrund der Gefahr, dass die Dateien gecached werden nicht für die Einbindung der Engineering Apps in Medias verwendbar. Auch bei der Variante „media“ wurden für die Dateien keine Hashes angezeigt, weshalb diese Option ebenfalls nicht in Frage kommt. Für die Varianten „all“

und „bundles“ wurden zwar Hashes generiert, diese sind jedoch nicht optional zu setzen, weshalb die Dateinamen ausgetauscht werden müssten und auch dieser Ansatz verworfen wurde. Daraus konnte man schließen, dass es also keine bereits in den Angular „build“ Befehl integrierte Variante gibt, die Hashes für die Dateien im Dist Ordner Optional zu setzen, weshalb eine andere Möglichkeit in Betracht gezogen werden musste [vgl. 4].

Daraufhin wurden nach einer nicht erfolgreichen online-Recherche mehrere Fachexperten aus dem Bereich Data Science Solutions bei Schaeffler befragt, wie man an das Problem herangehen kann. Die Antworten der Experten waren jedoch auch vage, da sich bisher noch nicht mit der Erstellung einer Web Component bei Schaeffler beschäftigt wurde und das Problem daher in dieser Form erstmalig auftrat.

Ein dabei in der Theorie behandelter Ansatz war die Verwendung von Webpack. Webpack ist ein Open Source Framework, das als Modul-Bundler für Java Script Anwendungen fungiert. Somit werden alle notwendigen Assets der Anwendung mittels Webpack in ein Bundle komprimiert, das dann in Medias eingebunden werden kann [vgl. 30]. Als Vorschlag zu einer möglichen Umsetzung kam dabei eine Variante dieses Bundle über ein in NGINX Plus integriertes webpack Modul zu generieren. Als dieses jedoch kurzfristig während eines Meetings mit dem Team vertestet wurde, zerstörte die Anwendung von Web Pack eine Test Web Component, indem diese beim Generieren des Bundles so lange brauchte, dass sie dabei mehrmals abstürzte und ein Entfernen des Webpacks aus der Anwendung aufgrund mehrerer automatisch eingefügter Abhängigkeiten so gut wie unmöglich wurde. Aus Sorge dies könnte auch bei den Engineering Apps geschehen, wurde der Ansatz schließlich als Haupt-Lösungsansatz verworfen.

Die Grundsätzliche Idee des Webpacks jedoch alle Dateien in eine zu bundlen, führte zu einer darauf folgenden Idee:

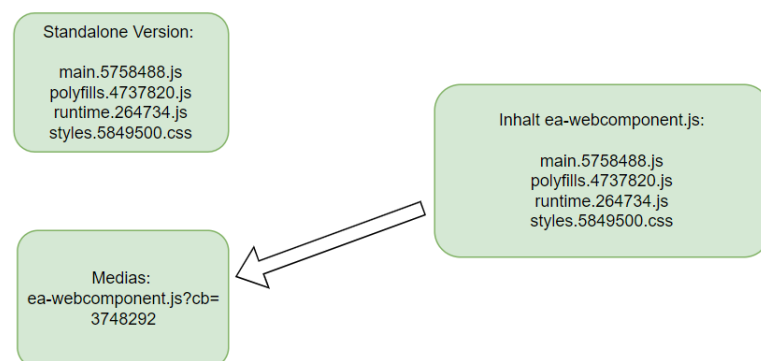


Abbildung 4.4: Konzept für die „ea-webcomponent“ Datei

Dieser in der Grafik dargestellte Lösungsansatz bestand letztendlich daraus, alle automatisch erzeugten Skripte - wie auch beim Webpack- in ein anderes Skript einzubauen, das beim

Build mit erzeugt wird und alle anderen Skripte enthält, inklusive der Hashes. Dazu sollte allerdings nicht Webpack verwendet werden, sondern anderweitig erstellt werden. An dieses Skript soll dann ein Cachebuster eingebaut werden, der die Zeit, die die Datei im Cache verweilt auf null Sekunden setzt, was zur Folge hat, dass es immer wieder vom Server neu angefragt wird. Dieses Skript funktioniert somit ähnlich wie die index.html, mit dem Unterschied, dass es auch in der Mediaseinbindung funktioniert.

Diese Datei muss anschließend dynamisch nach dem Build erstellt werden, weshalb beschlossen wurde, die Datei im Frontend Repository zu erstellen und in der Deployment Pipeline zu befüllen.

4.2.2 Mischung der Styles von Medias und Schaeffler

Ein weiteres Problem, das zufällig bei der Vertestung der Web Component identifiziert wurde und welches bei der Implementierung einer Web Component zur Einbindung in Medias auftreten kann, ist die Tatsache, dass die Styles der beiden Websites - sofern nicht vollständig voneinander isoliert - gegenseitig beeinflussen können. Dies könnte zur Folge haben, dass die Styles der Seiten aufeinander übergreifen. Ein Beispiel dafür ist etwa, wenn auf der Website, in die die Web Component eingebunden werden soll, eine Schriftfarbe gewählt wird die im gesamten „Body“ der Seite gilt. Wenn die Web Component dann im Body eingefügt ist, kann es vorkommen, dass diese - sofern keine eigene Schriftfarbe spezifiziert ist, ebenfalls die Farbe aus dem Body annimmt. Dies wird im folgenden anhand eines Testbeispiels aus der „test.html“ Seite und der zuvor gebauten Test Web Component erläutert, indem in die Datei „test.html“ ein Styletag in den Header eingefügt wird, der die Schriftfarbe von allem, was im Body steht auf rot setzt. Gleichzeitig wird im Body die Test Web Component eingebunden, was zu folgendem Ergebnis führt:

Engineering Apps
Das ist die Parent Seite
Inside the Web Component
Wieder die Parent Seite

Abbildung 4.5: Style Mischung der Parent Seite zur Web Component

Gleichzeitig kann das Problem jedoch auch von der Web Component ausgehen. Da zum Styling der Engineering Apps das Tailwind Plugin für Angular verwendet wird, wurde im Zuge dieser Demonstration ebenfalls Tailwind in die Test Web Component integriert. Tailwind ist dabei ein Design System das in verschiedene Frameworks integriert werden kann, und über zahlreiche vorgefertigte CSS Klassen verfügt [vgl. 27]. Wie auch die anderen genutzten Module, muss Tailwind zuerst per „npm install“ in der Konsole installiert werden,

eine config Datei per Konsolenbefehl erstellt werden und daraufhin in das CSS-Stylesheet der Web Component importiert werden mit dem Code:

```
1 @tailwind base;  
2 @tailwind components;  
3 @tailwind utilities;
```

Listing 4.7: Initialisierung von Tailwind in der Globalen CSS Datei

Danach sind die vorgefertigten Tailwind Klassen in der Web Component nutzbar. In der Test-Web Component wurde daraufhin mittels Tailwind Klasse hinter den Text ein grüner Hintergrund hinterlegt. Eine gleich benannte Klasse wurde dann ebenfalls in den H1 Tag der test.html Datei eingefügt.

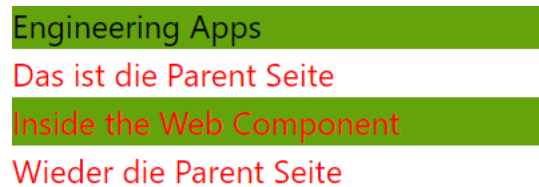


Abbildung 4.6: Beidseitige Style Mischung

Wie in der obigen Grafik zu sehen ist, wird die Tailwind Style Klasse auch auf die „zufällig“ gleich heiende Klasse angewendet, sowie im folgenden auf gleich benannte HTML Tags, wie der H1 Tag fr berschriften. Deshalb wird eine vollstndige Isolierung der Seiten voneinander in Betracht gezogen, damit sich die Seiten nicht gegenseitig beeinflussen. Diese Isolierung ist mit Hilfe einer bereits in Angular spezifizierten Methode namens View Encapsulation zu erreichen.

Laut Angular Dokumentation existieren dabei drei verschiedene Werte, die die View Encapsulation annehmen kann, Emulated, None und Shadow Dom.

Zuerst wurde die Art „Emulated“ zur View Encapsulation getestet, die fr viele Angular Installationen der Default Wert ist, sofern View Encapsulation in der Anwendungen existiert [vgl. 7]. Emulated stellt dabei sicher, dass die Styles, die einer Komponente zugewiesen werden auch nur fr die Komponente funktionieren, die Komponente allerdings immernoch von Global geltenden Styles betroffen ist. Dazu wurde in die Testkomponente in der Datei „app.component.ts“ der Parameter View Encapsulation importiert und anschlieend im „@Component“ Tag genutzt, indem die View Encapsulation hier auf Emulated gesetzt wurde [vgl. 7]. Das Ergebnis war jedoch, dass die Website immernoch identisch mit der Abbildung 4.6 aussah, und weshalb davon ausgegangen wird, dass durch den immerhin hnlichen Aufbau der Test App und der Engineering App seitens der Verwendung von Tailwind, das Ergebnis hnlich aussehen wrde.

„None“ hingegen ist der Wert, der gewählt wird, wenn keine Art von View Encapsulation gewünscht ist [vgl. 7]. Im Falle dieses Testbeispiels als auch im Fall der Engineering Apps sollen allerdings die Styles bewahrt werden, weshalb dieser Ansatz genau den gegenteiligen Effekt hat und somit nicht angewendet wird. In vielen Fällen ist dies auch der von Angular gesetzte Default.

Die dritte Art der View Encapsulation ist die Verwendung eines sogenannten Shadow Dom, um die Styles voneinander zu isolieren. Ein Shadow Dom ist dabei eine Variante, um ein separates Dom an ein Element anzufügen, das auch für sich selbst gestylt werden kann.

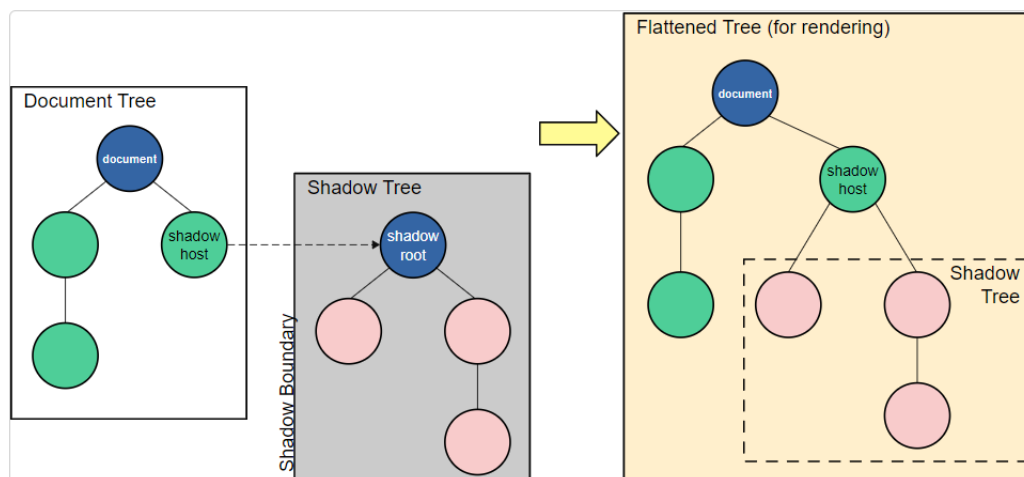


Abbildung 4.7: Aufbau einer Shadow DOM (Bildquelle: [24])

Es erlaubt also versteckten Dom Bäumen an Elemente in der regulären Dom angefügt zu werden, wie in der obigen Abbildung zu sehen ist, welche dann zu einem sogenannten Shadow-Root werden. Unter diesem befinden sich dann die versteckten DOM Bäume, die sogenannten Shadow Trees. An diese können dann letztendlich, wie auch an ein normales Dom, Elemente und Styles angehängt werden, die dann nur für die Shadow Dom verwendet werden können [vgl. 24].

Auch dieser Ansatz wurde in der Testkomponente verwendet, indem der Parameter Emulated durch den Parameter Shadow Dom ersetzt wurde, und lieferte das folgende Ergebnis:

Engineering Apps

Das ist die Parent Seite

Inside the Web Component

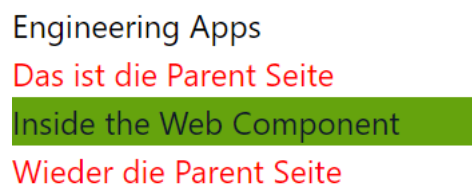
Wieder die Parent Seite

Abbildung 4.8: Style Mischung bei Verwendung einer Shadow Dom

Während also zwar die Schriftfarbe des Inhalts der Web Component nun nicht mehr von außen beeinflusst wird, wird ebenfalls die Hintergrundfarbe der Web Component ausgeschlossen und nicht mehr angewendet. Auf die HTML Seite jedoch, die die selbe allerdings undefinierte Klasse verwendet, wird ein grüner Hintergrund angewendet, obwohl er nicht in der Web Component verwendet wird und dies nicht gewünscht ist. Somit ist der Ansatz, dass man eine Shadow Dom nutzt ebenfalls im Falle der Engineering App nicht geeignet, zumal in diese laut jetzigem Konzept auch andere Assets geladen werden müssen, die aber nicht direkt im style.css stehen, was bedeutet dass die Shadow Dom diese auch aussperren würde, wie die Tailwind Klassen.

Da die bereits in Angular existierenden Lösungen nicht für den Use Case geeignet sind, wurde des weiteren eine weitere in Tailwind existierende Lösungsmethode getestet. Für diese wird ein selbst gewählter Prefix in der tailwind.config.ts gesetzt, im Fall des Testbeispiels das Prefix „tw-“ [vgl. 28]. Dieser Prefix wird anschließend vor alle Tailwind Klassen gesetzt, die in den HTML Bestandteilen der App verwendet werden. Aus der Klasse „bg-lime-600“ wird also beispielsweise „tw-bg-lime-600“.

Nachdem alle Tailwind Klassen im Testbeispiel also mit einem Prefix versehen wurden, wurde ein neuer Build der App gestartet, und in der Browseranzeige des HTML Dokuments zeigte sich folgendes Bild:



Engineering Apps
Das ist die Parent Seite
Inside the Web Component
Wieder die Parent Seite

Abbildung 4.9: Lösung des Styleproblems mittels einem Tailwind Prefix

Die Website wird korrekt angezeigt, indem die Klassen sich nicht mehr gegenseitig beeinflussen. Somit wäre dies auch für die Engineering App eine Möglichkeit, die Styles komplett voneinander zu isolieren. Seitens der Abteilung gab es allerdings bei der Vorstellung dieser Lösung Kritik an der Tatsache, dass das Prefix manuell in die verwendeten Tailwind Klassen eingefügt werden müsste und dies ein sehr großer Aufwand ist, der für jede neue Web Component wiederholt werden müsste, was in der Abteilung vorerst negativ aufgenommen wurde. Wenn eine vollkommen neue App gebaut werden würde, ist der Prefix allerdings eine gute Lösung um Style Mischungen zu verhindern, da er in diesem Fall keinen Mehraufwand mit sich bringt, wenn er von Beginn an eingebaut wird.

4.3 Konzept der Einbindung

Um die Engineering Apps zu einer Web Component umzubauen wurde letztendlich aufgrund von Zeitdruck seitens des Kunden Medias beschlossen, einige Teammitglieder der Abteilung Custom Data and AI2 im Zuge des nächsten Abteilungssprints mit einzubinden, um eine rechtzeitige Fertigstellung zu gewährleisten.

Zu diesem Team gehörten abgesehen vom Abteilungsbetreuer Robert Krause ebenfalls zwei andere Mitglieder, da sie einen guten Überblick über den Aufbau der Engineering Apps besitzen. Dieser ist komplex, da die Engineering Apps innerhalb Schaeffler auf mehrere Repositories verteilt sind, die innerhalb Schaeffler andere Berechtigungen benötigen und auch anderen Regeln unterliegen. Zu diesem Zeitpunkt stand bereits aufgrund der bisher gefunden Erkenntnisse fest, dass an mindestens zwei der Repositorys gearbeitet werden musste, damit die Umsetzung der Web Component funktioniert. Diese Repositories sind das Frontend Deployment Repository der Engineering App, und das sogenannte Monorepository, in dem der eigentliche Code zur Engineering App sich befindet zusammen mit vielen anderen Schaeffler Anwendungen.

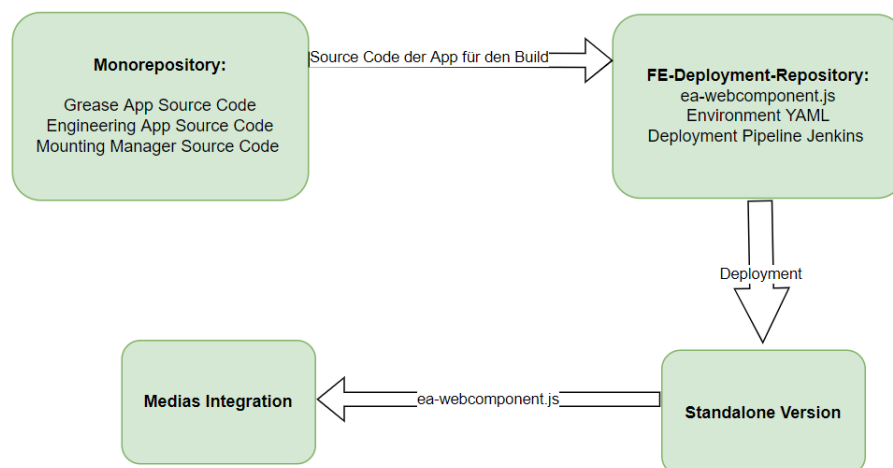


Abbildung 4.10: Übersicht über die Dateiverteilung

Aufgrund des komplexen Aufbaus der Gegebenheiten, wurden dementsprechend für den folgenden Scrum Sprint mit der Dauer von 2 Wochen Tasks erstellt, die zur Umsetzung der Web Component dienen und die verschiedenen Schritte darstellten.

Schritt 1 war dabei, der tatsächliche Umbau der Engineering App zur Web Component durch Einfügen der verschiedenen Code Bestandteile, während der Folgetask die Erstellung der Bundle-Datei war. Da diese Schritte in verschiedenen Repositories verändert werden mussten, bestand die Möglichkeit die Tasks parallel auszuführen. Auch musste ein Guide für die Mediasentwickler zur Einbindung verfasst werden, einige weitere Dateien angepasst

werden, eine Dokumentation für die zukünftige Replizierung des Vorgehens verfasst werden und natürlich die Einbindung auch praktisch im Browser vertestet werden, um sicher zu stellen, dass dies funktioniert. Mit diesen Tasks wurde dann in den Sprint gestartet.

5 Umsetzung der Engineering App als Web Component

In Kapitel 5 dieser Arbeit wird die Engineering App zu einer funktionierenden Web Component umgebaut. Dazu werden die einzelnen Schritte und Hindernisse in den verschiedenen Kapiteln beleuchtet. Besonderen Wert wurde dabei auf den initialen Umbau der Engineering App sowie die Pipelineanpassungen mittels Jenkins gelegt, da diese Schritte essentiell sind. Ebenso wird das Problem der Style Mischung gelöst und weitere Anpassungen werden getätigt, bevor die Web Component evaluiert wird und anschließend mögliche Weiterentwicklungen analysiert werden.

5.1 Umbau der Engineering App zur Web Component

Um die Webcomponent auch für die Engineering App zu erstellen, muss in die sich im Monorepository befindenden Dateien der Engineering App der in Kapitel 4.1 festgelegte Code eingefügt und für die Zwecke der Engineering App angepasst werden. Dazu müssen vor allem die Module „createCustomElement“ und „Injector“ in die „app.module.ts“ importiert werden, bevor - wie auch in der Test Version - der folgende Code in die Export Class eingefügt werden muss.

```
1 constructor(private injector: Injector) {  
2     const el = createCustomElement(AppComponent, { injector  
3         });  
4     customElements.define('engineering-app', el);  
5 }  
6 ngDoBootstrap() {}
```

Listing 5.1: Umbau der Engineering App zur Web Component

Beim Einfügen dieses Code ist es wichtig, zu überprüfen, ob in der selben Datei nochmals ein Bootstrapping Modul verwendet wird. Sofern dies der Fall ist, wird laut späteren Tests zur Evaluierung die App, sobald sie gebaut wird, zweimal initialisiert, einmal als Web Component und einmal als normale Angular Anwendung. Die normale Angular Anwendung, die zeitgleich erstellt wird, sorgt allerdings für Fehler und soll deshalb nicht initialisiert werden, sondern ausschließlich die Web Component. Die Bootstrapping Funktion die somit gebraucht wird, ist die neu hinzugefügte in der Export Klasse und die andere muss gelöscht werden.

Daraufhin muss der Selector in der @Component Klasse in der Datei „app.component.ts“ verändert werden und auf den selben Namen gesetzt werden wie das Custom Element, im

Fall der Engineering App also „engineering-app“. Wenn dies nicht geschieht, tritt im Browser ein sogenannter App Root Error auf, der darauf hinweist, dass das sogenannte „Root Element“ der Anwendung nicht existiert. Dies führt ebenfalls, wie in eigenen praktischen Tests zu sehen war, zu Fehlern bei der Anzeige der Web Component im Browser, was zu vermeiden gilt.

Im Fall der Engineering Apps jedoch müssen auch Daten an die App übergeben werden, die zur Berechnung der Zielwerte benötigt werden. Somit müssen auch hier wieder wie in der Test Anwendung Input Parameter gesetzt werden, an die später die Werte übergeben werden können. Dazu muss ebenfalls in der „app.component.ts“ das Modul „Input“ importiert werden und schließlich die verschiedenen benötigten Inputparameter in der Export Klasse der Datei definiert werden.

Im Fall der Engineering App sind das die Daten, die benötigt werden um die Berechnungen tätigen zu können, also die „wt-part-id“, die Sprache und die „bearing designation“, die aus Medias übernommen werden sollen. Diese sind dafür zuständig, dass durch die Lagerbezeichnung die zugehörigen Werte gefunden werden und die Einbindung auch funktioniert.

Anschließend sind die Berechnungen der Engineering App auch in der Einbindung funktionsfähig.

5.2 Anpassungen der Jenkins Pipeline

Um die Engineering App in Medias einbinden zu können ohne bei jedem Updates die Skripte tauschen zu müssen und trotzdem zu garantieren, dass immer die neuste Version geladen wird, müssen einige Änderungen für die Einbindung geschaffen werden. Dies soll geschehen, indem die benötigten Dateien aus dem „dist“ Ordner alle aus einer Datei referenziert werden sollen, die mit einem „Cachebuster“ versehen werden soll, sodass diese Datei nicht gecached wird, und die referenzierten Dateien immer die neuen korrekten Dateien sind.

Da die Änderungen laut Fachexperten am besten vor dem Deployment, aber nach dem Build geschehen sollen, finden diese innerhalb der Deployment Pipeline der Engineering App statt, welche in einem eigenen Github Repository geregelt ist. Die Pipeline selbst verwendet Jenkins, einen Open Source Automatisierungsserver, der zum Build, zum Deployment und zur Automatisierung vieler Projekte verwendet wird [vgl. 18].

Der erste Schritt, der benötigt wird um die Dateien zu einem zu verbinden, ist die Erstellung der Enddatei mit dem Namen, die es haben soll. Im Fall der Engineering App ist dies der Name „ea-webcomponent.js“. Dieses wird auf der selben Organisationsebene wie die Deployment Pipeline erstellt.

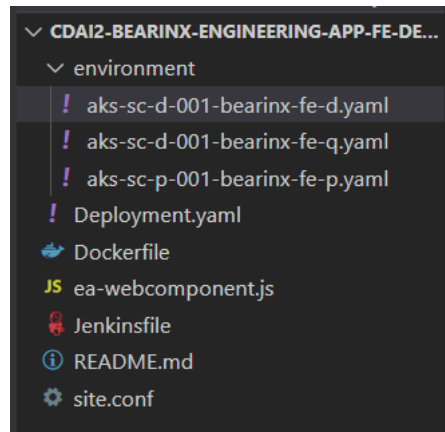


Abbildung 5.1: Organisationsstruktur der Deployment Pipeline

Wie in Abbildung 5.1 zu sehen ist, existieren in dem Repository ebenfalls drei „yaml“ Dateien im „Environment“ Ordner. Diese Dateien existieren, um die Rahmenparameter für die verschiedenen Deployment Umgebungen zu setzen. Diese unterscheiden sich im Namen vor allem am letzten Buchstaben. Die .yaml Datei mit „d“ am Ende ist für das Development Environment verantwortlich, das bei Schaeffler genutzt wird um Änderungen und neue Features zu testen, während man entwickelt.

Das „q“-Environment hingegen, existiert, damit nach der ersten Entwicklung der neue Stand probeweise veröffentlicht werden kann, um diesen Stakeholdern und Fachexperten zu zeigen, die Änderungen aber noch nicht im Produktivsystem zu haben. Wenn diese Änderungen von allen angenommen wurden und umfangreichen Tests unterzogen wurden, werden die Änderungen in das letzte Environment, das „p“-Environment, also die Produktivumgebung, übernommen. Diese Dateien sind vor allem wichtig, weil in ihnen ein neuer Parameter gesetzt werden muss, damit die „ea-webcomponent.js“ Datei später auch von der Medias Einbindung im Browser gefunden werden kann.

Damit dies möglich ist, muss hier in jedem der drei Umgebungsdateien der Parameter „host_url“ gesetzt werden, jeweils mit der richtigen URL wohin die Datei deployed werden soll.

```

1 #in aks-sc-d-001-bearinx-fe-d.yaml
2 host_url: https://engineeringapp-d.dev.dp.schaeffler/
3
4 #in aks-sc-d-001-bearinx-fe-q.yaml
5 host_url: https://engineeringapp-q.dev.dp.schaeffler/
6
7 #in aks-sc-d-001-bearinx-fe-p.yaml
8 host_url: https://engineeringapp-p.dev.dp.schaeffler/

```

Listing 5.2: Setzen der Host URL in den verschiedenen Deployment Umgebungen

Nachdem dies erledigt ist, kann die zu diesem Zeitpunkt leere „ea-webcomponent.js“ Datei

geöffnet werden, in welche schließlich die richtigen Dateipfade für die im Build der App entstandenen Dateien automatisch ausgetauscht werden können sollen.

Dazu müssen jedoch zuerst Platzhalter für die Dateien angelegt werden. Dies geschieht indem für die drei benötigten Java-Script Dateien sogenannte Skript-Variablen angelegt werden, die später durch die richtigen Dateipfade ersetzt werden.

```
1 var script = document.createElement('script');
2 script.src = 'RUNTIME_FILE';
3 document.head.appendChild(script);
4
5 var script = document.createElement('script');
6 script.src = 'POLYFILLS_FILE';
7 document.head.appendChild(script);
8
9 var script = document.createElement('script');
10 script.src = 'MAIN_FILE';
11 document.head.appendChild(script);
```

Listing 5.3: Erstellung der Platzhalter für die Javascript Dateien

Dies geschieht, wie im obigen Code zu sehen ist, innerhalb von drei Zeilen pro Skript, in denen zuerst die Variable definiert wird, dann der Platzhalter generiert wird und diese schließlich oben an das am Ende fertige Dokument angehängt wird.

Für das Style Skript ist das Konzept ähnlich, aber nicht identisch, da hier eine andere Variable verwendet werden muss, da Stylesheets anders importiert werden müssen. Anstatt einer „script“ Variable muss an dieser Stelle eine „link“ Variable verwendet werden. Dabei ist es besonders wichtig, dass die dazugehörige „media“ in diesem Fall auf „all“ gesetzt wird, da das Stylesheet sonst möglicherweise nicht angewendet werden kann.

Im Fall der Engineering App sieht der Platzhalter für das Stylesheet so aus:

```
1 var link = document.createElement( "link" );
2 link.href = "STYLE_FILE";
3 link.type = "text/css";
4 link.rel = "stylesheet";
5 link.media = "all";
6 link.onload = () => this.media='all';
7 document.head.appendChild( link )
```

Listing 5.4: Erstellung des Platzhalters für das Stylesheet

Anschließend sind alle Platzhalter erstellt und man kann zur Jenkinsdatei übergehen, in welchem das Deployment geregelt wird.

Diese Datei besteht aus einer Deployment Pipeline, die aus mehreren Stages besteht, welche anschließend auch in Jenkins eingesehen werden können. Die Stages in der Pipeline sind dabei die folgenden:

- Preperation
- Artifactory
- Docker Build
- Trivy Scan Container Image
- AKS Deployment
- Docker Registry Cleanup

Um hier die Dateinamen aus dem „dist“ Ordner in die „ea-webcomponent.js“ Datei einzusetzen, wird hier eine neue Stage generiert, zu der die Fachexperten aus der Abteilung meinten, dass diese am besten zwischen den bereits existierenden Stages Artifactory und Docker Build liegen sollte, da im Schritt Artifactory der „dist“ Ordner entpackt wird und danach das Docker Image für das Deployment generiert wird, in den die Datei im besten Fall schon eingefügt werden soll.

Diese Stage wurde im Fall der Engineering App „File Build“ genannt und hat die Voraussetzung, dass zwei Parameter existieren müssen die in den vorhergehenden Stages generiert werden, da die Ergebnisse der Stages benötigt werden. Wenn die Stages vorher nicht ausgeführt wurden, wird somit diese Stage ebenfalls übersprungen.

```

1 MAIN_SCRIPT= sh (
2     script: "ls dist | grep main.*js\$",
3     returnStdout:true
4 ) .trim()
5 RUNTIME_SCRIPT= sh (
6     script: "ls dist | grep runtime.*js\$",
7     returnStdout:true
8 ) .trim()
9 POLYFILLS_SCRIPT= sh (
10    script:"ls dist | grep polyfills.*js\$",
11    returnStdout:true
12 ) .trim()
13 STYLE_SCRIPT= sh(
14    script:"ls dist | grep style.*css\$",
15    returnStdout:true
16 ) .trim()
17 echo "${MAIN_SCRIPT} ${RUNTIME_SCRIPT} ${POLYFILLS_SCRIPT} ${
    STYLE_SCRIPT}"

```

Listing 5.5: Abfrage der Java-Skript Dateien mithilfe von Regular Expressions

Innerhalb der Stage werden dann neue Variablen definiert, in die dann mittels Shell Comands die bereits in der Artifactory Stage entpackten Dateien hinterlegt werden. Um dies zu bewerkstelligen wurden sogenannte regular expressions verwendet, die mittels eines Generators auf der Seite „<https://regex101.com/>“ erstellt wurden. Eine Regular Expression ist

dabei eine Zeichenkette aus verschiedenen Operatoren und Bezeichnungen, die dazu dient den Input Text zu matchen, und für den Fall der Engineering Apps diese innerhalb der Variable zurückzugeben [vgl. 21].

Daraufhin müssen die nun in den neuen Variablen hinterlegten Skripte in die „ea-webcomponent.js“ Datei an die Stelle der dort hinterlegten Skripte gesetzt werden. Dies geschieht mithilfe von Kommandozeilenbefehlen, die dafür sorgen, dass die Skripte an den richtigen Stellen ausgetauscht werden und schließlich die „ea-webcomponent.js“ Datei in den „dist“ Ordner kopiert wird.

```

1 sh """
2     sed -i "s#MAIN_FILE#${clusterEnvs.host_url}${
3         MAIN_SCRIPT}#g" ea-webcomponent.js
4     sed -i "s#RUNTIME_FILE#${clusterEnvs.host_url}${
5         RUNTIME_SCRIPT}#g" ea-webcomponent.js
6     sed -i "s#POLYFILLS_FILE#${clusterEnvs.host_url}${
7         POLYFILLS_SCRIPT}#g" ea-webcomponent.js
8     sed -i "s#STYLE_FILE#${clusterEnvs.host_url}${
9         STYLE_SCRIPT}#g" ea-webcomponent.js
10 """
11 sh "cp ea-webcomponent.js dist/ea-webcomponent.js"

```

Listing 5.6: Ersetzung der Skripte innerhalb der Datei „ea-webcomponent.js“

Dies wird nun bei jedem neuen Build der App durchgeführt, was dazu führt, dass nun die neuen Dateien mit den Hashes immer in der Java Skript Datei zu finden sind. Für diese wird anschließend an die Änderungen in der Jenkins Pipeline in einem weiterführenden Sprint ein Cachebuster generiert, der verhindert, dass diese Datei gecached wird, was dazu führt, dass es bei jedem Aufruf der Seite neu geladen werden muss. Dadurch erhält der Nutzer immer die neuste Version der Website und innerhalb der Medias einbindung müssen keine Dateinamen geändert werden.

5.3 Style Isolation der Web Component

Dennoch bleibt das bereits identifizierte Problem, dass die Styles von Medias sich mit den Styles der Web Component mischen könnten. Vorübergehend wurde das Problem dadurch gelöst, dass innerhalb der Web Component die eigenen Styles auf „important“ gesetzt wurden, was bedeutet, dass diese vorrangig vor den Medias-Stylesheets behandelt werden. Um diese Styles wurde schließlich ein tag „engineering-app{}“ gesetzt, der wie ein Prefix agiert und dafür sorgt, dass der Style nur innerhalb der Engineering App greift. Da dies jedoch nur eine vorübergehende Lösung des Problems darstellt und nach mehreren Gesprächen mit den Angular Experten in der Abteilung keine zufriedenstellende Lösung gefunden werden

konnte, wurde sich doch entschieden die bereits existierende Lösung mittels eines Prefixes umzusetzen, damit die Styles der Webcomponent und von Medias doch vollständig voneinander isoliert sind. Dies soll in einem weiterführenden Sprint von einem Mitglied der Abteilung umgesetzt werden.

Zusätzlich wurden bereits Überlegungen getätigt, eine einheitliche Benennung dieses Prefixes für alle Engineering Apps zu generieren, und eine Variation dieses Prefixes auf alle Engineering Apps anzuwenden, um zukünftige Styleprobleme zu vermeiden. Der Grund dafür war hauptsächlich, dass während der Vertestung der Web Component immer mehr Styleprobleme auftraten, die schließlich auch den Medias Verantwortlichen aufgefallen waren, was dazu führte, dass auch diese eine strikte Styletrennung zwischen der Web Component und ihrer Seite wünschten.

5.4 Weitere Anpassungen an der Engineering App

Bei weiteren lokalen Tests trat allerdings eine weitere Fehlermeldung auf, ein sogenannter CORS „Access Control Allow Origin“ Error, der die Anzeige der testweise eingebundenen Web Component zu verhindert hat. Ein CORS (Cross Origin Resource Sharing) - Error tritt auf, wenn die verwendeten Ressourcen nicht von den selben Quellen kommen, und dies auch nicht explizit innerhalb der CORS Policy zugelassen wurde [vgl. 23]. Im speziellen Fall der Engineering App tritt dieser Error auf, da die Engineering App zuerst eigenständig mit allen Ressourcen deployed wird, und danach als Web Component in die Medias Plattform eingebunden wird. Dabei versucht der Medias Header also auf Schaeffler Ressourcen zuzugreifen, was aber nicht explizit zugelassen ist. Deshalb wird der Zugriff der Medias Integration auf die Ressourcen der Engineering App verhindert.

Um diesen Fehler zu umgehen, muss Medias in der CORS Policy zugelassen werden und auf eine sogenannte „Whitelist“ gesetzt werden, von Seiten die auf die Ressourcen zugreifen können. Als dies erkannt wurde, wurden die Medias URLs für die Engineering Apps zugelassen.

Ebenso entstand auch ein Problem mit den relativen Pfaden der Engineering App, durch die Ressourcen nachgeladen werden und somit wurden einige Styles der Engineering App nicht korrekt nachgeladen. Um das zu beheben, müssen die relativen Dateipfade durch absolute Pfade ersetzt werden, damit die Ressourcen der Engineering App auch korrekt nachgeladen werden können, obwohl die App eingebunden ist. Dazu wurden verschiedene in Angular enthaltene Optionen überprüft und zwei Befehle gefunden, die potenziell Anwendung finden könnten.

Der erste davon war der sogenannte „base-href“ Befehl. Dieser sorgt laut Angular Dokumentation dafür, dass ein Pfad festgelegt wird, um Bilder, Styles und andere Ressourcen über

diesen Pfad nachzuladen [vgl. 5]. Bei ersten Tests dieses Ansatzes wurden allerdings keine zufriedenstellenden Ergebnisse erzielt, da die Icons des Stylesheets nicht korrekt geladen wurden, weshalb nach einer Alternative gesucht wurde.

Diese Alternative bestand aus der Nutzung des ebenfalls in Angular integrierten Features „deploy-url“. Diese hat die selbe Funktion wie die „base-href“ Option, indem beide für das Laden von Styles, Bildern und anderen CSS-Ressourcen verwendet werden können [vgl. 5]. Bei der Testung der deploy-url Funktion wurde allerdings festgestellt, dass die Icons der Berechnung sowie der Rest der Styles korrekt in die Anwendung geladen wurden.

Die Codeeinbindung funktioniert im Fall der Engineering App so, dass in der „project.json“ Datei der App jeweils für den lokalen Build, den Development Build, den QA Build und den Production Build der Parameter „deploy-url“ eingesetzt wird.

Für den Production Build sieht der Code dann in etwa so aus:

```
1 "production": {
2     "outputHashing": "bundles",
3     "deployUrl": "https://engineeringapp.dp.schaeffler/",
4
5     "optimization": {
6         "scripts": true,
7         "styles": {
8             "minify": true,
9             "inlineCritical": false
10        },
11    },
12    "fonts": false
13 }
```

Listing 5.7: Verwendung des Parameters „deploy-url“

Diese Lösung birgt allerdings ein Problem, wenn man weiter in die Zukunft denkt: Die Funktion Deploy Url ist laut Angular „deprecated“ und wird in einer der zukünftigen Angular Versionen entfernt. Die Alternative, die Angular dabei vorschlägt ist dabei die bereits genannte „base-href“, die allerdings im Fall der Engineering App nicht funktioniert. Somit ist dies nur eine kurzfristige Lösung um die Styles in der Web Component nachzuladen, für die in Zukunft eine Alternative geschaffen werden muss [vgl. 4]. Auch musste der Parameter „Inline Critical“ für Styles in der Datei „project.json“ deaktiviert werden, damit die Einbindung funktionierte.

Ebenfalls mussten für die Übersetzungsskripte der Engineering App, die dafür sorgen, dass die App in vielen Sprachen verfügbar ist und automatisch die bei Medias hinterlegte Sprache verwendet wird, angepasst werden. Da diese Dateien allerdings in einem Teil des Repositories lagen, auf das die Verfasserin keinen Zugriff hatte, wurde dies von einem anderen Teammitglied erledigt.

Daraufhin konnte die Einbindung im Browser lokal getestet werden, indem das Netzwerk-analysetool des Browsers auf der Medias Seite verwendet wurde. Dazu wurde der iFrame Tag in dieser Ansicht durch die Web Component inklusive der Skripte ersetzt. Die Web Component wurde daraufhin richtig angezeigt und somit zum Test für die Medias Entwickler freigegeben.

5.5 Dokumentationen zur Einbindung der Web Component

Damit die Entwickler von Medias die Web Component auch umfangreich testen können, wurde diese in das QA Environment von Schaeffler deployed und ein Leitfaden zur Einbindung der Web Component für die Medias Entwickler verfasst. In diesem wurde spezifiziert, wie die Web Component in Medias zu integrieren ist.

Der erste Schritt dafür war, dass der iFrame Tag auf der Medias Seite entfernt werden muss, inklusive aller Inhalte die darin vorhanden sind. Dies schließt auch alle im `<iframe>` spezifizierten Styles mit ein, da diese aufgrund des in das `ea-webcomponent.js` integrierten Stylesheets überflüssig geworden sind. Anschließend muss der folgende Code an die Stelle des `iframe` eingesetzt werden, um die Web Component zu integrieren:

```
1 <engineering-app wt-part-id="" bearing-designation="" language=
  ""></engineering-app>
2 <script src="https://engineeringapp-q.dev.dp.schaeffler/ea-
  webcomponent.js"></script>
```

Listing 5.8: Integration der Web Component in Medias

Die im Web Component tag spezifizierten Input Parameter werden dabei automatisch übergeben und die Einbindung ist bereit zur Testung.

Parallel wurde von Schaeffler Seiten auch noch eine Dokumentation zur Erstellung einer Web Component für die Engineering Apps im abteilungsinternen Confluence Bereich angelegt, um sicherzustellen, dass das neu gefundene Know How auch festgehalten wird und für zukünftige Use Cases genutzt werden kann.

5.6 Evaluierung der Webcomponent

Um die Web Component für die Engineering App angemessen evaluieren zu können, muss zuerst sichergestellt werden, dass die zu Beginn definierten Kriterien für die Medias Einbindung der Engineering Apps auch erfüllt wurden.

Kurz nach der Übergabe des Einbindungsleitfadens für die Medias Entwickler kam von deren Seite zurück, dass die Einbindung der Engineering App auch von deren Seite nach mehreren Testreihen problemlos funktioniert, weshalb die Funktionalität der App beidseitig gewährleistet ist. Dadurch ist das erste Kriterium für die Einbindung erfüllt.

Auch wurde die Einbindung der Engineering App in die Medias Seite mittels der Analysefunktion der Browser lokal getestet, und die Web Component wurde bei diesen Tests korrekt von allen getesteten Browsern angezeigt. Dies umfasste abermals Google Chrome, Microsoft Edge, Firefox und Safari.

Durch die Lösung alle Skripte immer aktuell einzubinden, indem sie in ein Skript eingefügt werden, dass auf sie verweist, welches mit einem Cachebuster ausgestattet ist, wurde eine elegante Lösung geschaffen die Engineering App sowohl in der Standalone Version als auch in der eingebundenen Variante immer aktuell zu halten. Dabei müssen jedoch aufgrund dieses Workarounds die Dateien nicht ausgetauscht werden, weshalb von Seiten von Medias kein zusätzlicher Aufwand für Hotfixes oder ähnliches anfällt. Dadurch, dass hier ebenfalls nur die Standalone Version in Medias integriert wird. Dadurch ist immer die aktuelle Version der Engineering App in Medias eingebunden, weshalb auch diese Kriterien erfüllt sind.

Ebenfalls sind theoretisch laut der Angular Dokumentation Möglichkeiten für User Tracking nun leichter umsetzbar, da die Web Component auch von Crawlern gesehen wird, und die Seite sowohl in der Standalone Version als auch in der Einbindung eine höhere SEO Bewertung gewinnt, da die Seite als eine Einheit auftritt. Dadurch können nun in der Theorie auch die Cursorbewegungen in der Web Component erfasst werden.

Da viele der Styles nur noch in der Engineering App selbst spezifiziert werden, können hier auch optische Anpassungen leichter vorgenommen werden, und das Design somit visuell optimiert werden. Aufgrund des Zeitmangels bei der Umsetzung ist dies allerdings - ebenso wie die Tracking Möglichkeiten - noch nicht geschehen, wobei zwischen der Einbindung per iFrame und der Einbindung per Web Component bereits geringe optische Unterschiede erkennbar sind.

Während viele visuelle Unterschiede der Versionen auch mit der Weiterentwicklung der App zu tun haben, die sich von einer Katalogberechnung der Bearingx Kollegen zu einer Kooperation zwischen Bearingx Backend und Custom Data and AI 2 Frontend entwickelt hat, ist dennoch erkennbar, dass die Verwendung einer Web Component visuelle Unterschiede mit sich bringt. Während Features wie einklappbare Parameter und der hinzugefügte CO2 Calculator zwar auf die weiterentwicklung zurückzuführen sind, existieren dennoch Unterschiede die nur durch die neue Einbindungsmethode bestimmt sind.

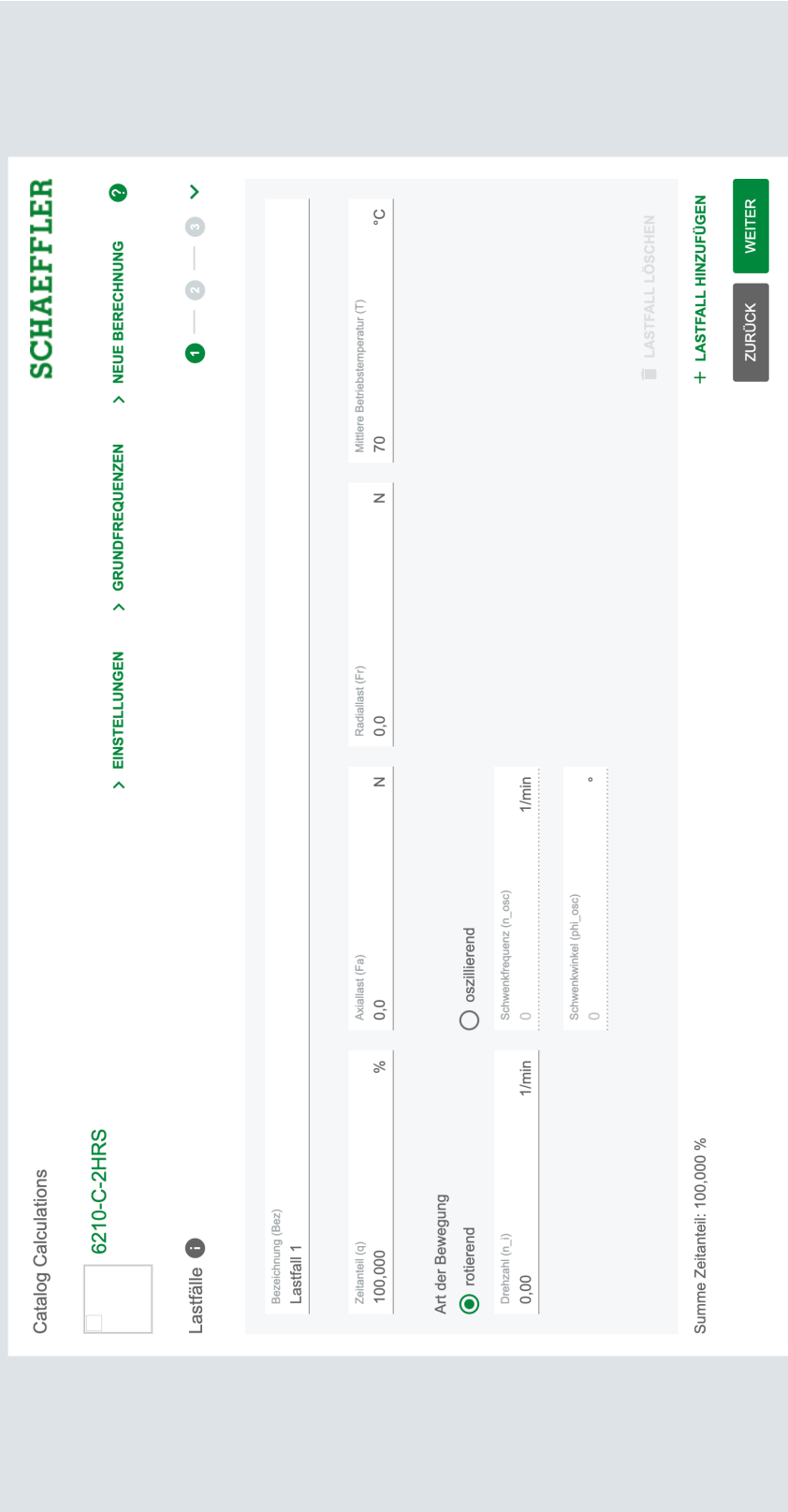


Abbildung 5.2: Einbindung der Engineering App in Medias mittels iFrame

Engineering App

Operating conditions for bearing 6210-C-2HRS

RESET INPUTS

Loads
* required information

Radial load * N

Axial load * N

Rotational speed
* required information

Rotational speed * 1/min

Choose your calculations

☒ Calculations

☒ CO₂ emissions

☐ Overrolling frequency

BASIC FREQUENCIES

Please add the remaining conditions.

Total value of CO₂

Production	Operation
1.33 kg	

SHOW REPORT

Abbildung 5.3: Einbindung der Engineering App in Medias mittels Web Component

Wie in Abbildung 5.2 zu sehen ist, hatte die Einbindung der Katalogberechnung zuvor einen grauen Hintergrund in Medias, der um die App herum zu sehen war. Dieser Hintergrund, der auf eine falsche Skalierung des iFrame zurückzuführen ist, ist mit der Einbindung der Engineering App als Web Component komplett verschwunden. Diese passt sich somit dem Ursprungsdesign der Standalone Version an. Ebenfalls ist zu beobachten, dass die sich durch die Einbindung mittels Web Component die Einbindung von der Höhe an die App angepasst hat, sodass der weite, nicht skalierbare Rand um die Einbindung bereits jetzt verschwunden ist.

Zudem ist der optische Unterschied auch dadurch erkennbar, dass die Qualität der Einbindung durch die Verwendung der Web Component massiv gestiegen ist. Während die hellen Schriftbestandteile trotz mehreren Anzeigeversuchen auf fast allen Bildschirmen verschwommen und nicht klar lesbar erschienen, sind auch die hellen Textbestandteile in der Web Component gut lesbar abgebildet. Das lässt bereits jetzt die Einbindung professioneller und optimierter wirken.

5.7 Möglichkeiten zur Weiterentwicklung der Web Component für die Engineering App

Wie bereits im vorherigen Kapitel angemerkt, gibt es dennoch Möglichkeiten zur Weiterentwicklung und weitere Schritte, die getroffen werden müssen, damit das Potenzial der Web Component vollständig ausgeschöpft werden kann. Dass diese Möglichkeiten nach wie vor nicht realisiert sind, liegt vor allem daran, dass die Zeit für die Umsetzung stark begrenzt war, und sich somit zuerst um die Funktionalität und die wichtigsten Features gekümmert wurde.

Eine wichtige Weiterentwicklung der Web Component ist es beispielsweise, die Styles der Website komplett responsive zu gestalten, sodass auf die Mediaeinbindung von jeglichen Geräten zugegriffen werden kann. Bisher beim iFrame war das anhand der Beschaffenheiten des iFrames nicht möglich, da hier Parameter wie die Dimension des iFrames in Medias fest einprogrammiert waren. Durch die neue Einbindung durch die Web Component entstehen jedoch neue Möglichkeiten, das Styling einfacher responsive zu gestalten. In der Abteilung wurden bereits erste Überlegungen angestellt, die Positionierung der Elemente anders zu gestalten, um leichter ein neues responsive Design für die Engineering App zu erstellen. Bei diesen Überlegungen wurde vor allem bisher darauf geachtet, die Engineering App auf eine Vielzahl von Geräten anzupassen, das Design muss jedoch im weiteren Verlauf noch überarbeitet und finalisiert werden, und schließlich in die Engineering App eingearbeitet werden.

Ein weiteres Thema, das weitere Nachforschungen bedarf, ist die Tracking Thematik. Bisher wird bei Medias für das Tracking Google Analytics verwendet und in der Standalone Version Application Insight. Es wurden jedoch seitens der Abteilung bereits Überlegungen angestellt, das Thema Tracking neu zu überdenken, da sich hier dafür, dass Tracking durch eine Web Component erleichtert werden soll, nur auf Theoretische Aussagen berufen wird und dieser Ansatz nie praxtisch vertestet wurde. Somit besteht bei diesem Thema die Möglichkeit einer weiteren studentischen Arbeit, die sich damit auseinandersetzen könnte, wie Nutzertracking sowohl in der Standalone Version als auch in der Medias Einbindung genutzt werden kann, und diese Ergebnisse anschließend praktisch auf die Engineering App anwenden.

Auch muss innerhalb der Engineering App immernoch eine strikte Styletrennung eingeführt werden, da die Styles bis zum jetzigen Zeitpunkt immernoch provisorisch voneinander getrennt existieren, und somit die Lösung mit dem Tailwind Prefix in das Programm eingearbeitet werden muss. Dazu wurde bereits ein Ticket geschrieben, und diese Aufgabe wird vermutlich in einem der nächsten CDAI2 Sprints erledigt werden.

6 Abschlussbetrachtung

In diesem Kapitel wird das Endergebnis der Arbeit kritisch betrachtet und ein Ausblick gegeben, ob das Thema Web Components sowohl von Medias als auch bei Schaeffler weiter verfolgt wird.

6.1 Ergebnis

Abschließend ist festzustellen, dass im Zuge dieser Arbeit das iFrame für eine der Engineering Apps durch eine Web Component ersetzt wurde, die auch bis auf einige noch ausstehende Punkte, vollständig funktionsfähig in Medias funktioniert. Dazu wurden mehrere Anpassungen in der Engineering App und den anderen Schaeffler Repositorys getätigt, um die Web Component lauffähig zu machen. Dieser Prozess war von hoher Komplexität, da für die Umsetzung mehrere Repositorys innerhalb Schaeffler beeinflusst werden mussten und viele Hindernisse auftraten, für die sowohl innerhalb Schaeffler als auch außerhalb des Unternehmens nur selten bereits existierende Lösungen gefunden werden konnten und daher oft Workarounds geschaffen werden mussten, um das Ziel zu erreichen. Beispiele für so einen Workaround sind beispielsweise der gewählte Style-Prefix oder die „ea-webcomponent.js“ Datei, die sowohl verhindern soll, dass Dateinamen ausgetauscht werden müssen, als auch dass die Anwendung veraltet. Dennoch konnte für fast alle Probleme eine angemessene und langanhaltende Lösung identifiziert werden, weshalb die Anwendung zukunftssicher ist und das Schema auch auf die anderen Cloud Lösungen von Schaeffler angewendet werden kann.

Um dies zu garantieren wurde eine Dokumentation geschaffen, die helfen soll das generierte Know How zu erhalten, da diese Arbeit auch Wissensaufbau in der Abteilung genutzt wurde. Zum bisherigen Zeitpunkt ist auch der Vertreter von Medias, der die Entwicklung mit Zeitdruck vorangetrieben hatte, mit der Anwendung zufrieden, da diese fristgerecht an Medias geliefert wurde und nach vielen Tests schließlich in Medias eingebunden wird. Somit ist die Engineering App bald als Web Component in der Produktivumgebung von Medias verfügbar. Dadurch werden in Zukunft die Medias Kunden ein besseres optisches Bild der Engineering App zu sehen bekommen.

Der Aufwand der Implementierung der Web Component gegenüber dem Aufwand der weiteren Nutzung der iFrame Technologie war höher als ursprünglich von sowohl Medias als auch CDAI2 vermutet, da viel Pionierarbeit damit verbunden war. Dies lässt sich vor allem auf die nur rudimentären Dokumentationen zurückführen, die für viele der genannten Herausforderungen unzureichend waren, weshalb nach anderweitigen Lösungsmustern gesucht werden

musste. Diese beruhten in vielen Fällen zuerst auf einer Suche nach Lösungsmustern innerhalb der vorgefundenen Dokumentationen. In vielen Fällen reichte das jedoch nicht aus, weshalb viele Angularexperten bei Schaeffler zeitweise zur Beratung hinzugezogen werden mussten, wie etwa bei der Lösung der Caching Thematik. Die Experten konnten jedoch aufgrund mangelnder Anwendungsfälle von Web Components auf komplexe Anwendungen wie die Grease App auch oft nicht mit Lösungen dienen, weshalb die Umsetzung der Web Component und die Bewältigung der genannten Hindernisse oftmals mit „Trial and Error“ verbunden waren. Deshalb sind viele der geschaffenen Workarounds weder zeitlich noch programmiertechnisch effizient gelöst worden. Aufgrund des Zeitdrucks von Seiten der Medias Vertreter blieb jedoch auch keine Zeit nach anderen geeigneten Lösungen zu suchen, weshalb die beschriebenen Workarounds vorerst bestehen bleiben.

Trotz der genannten Schwierigkeiten wurde aber die Engineering App in eine funktionierende Web Component umgebaut, die in Zukunft in Medias deployed wird, was sowohl laut der Abteilung Custom Data and AI 2 als auch laut Medias als Erfolg zu verzeichnen ist.

6.2 Ausblick

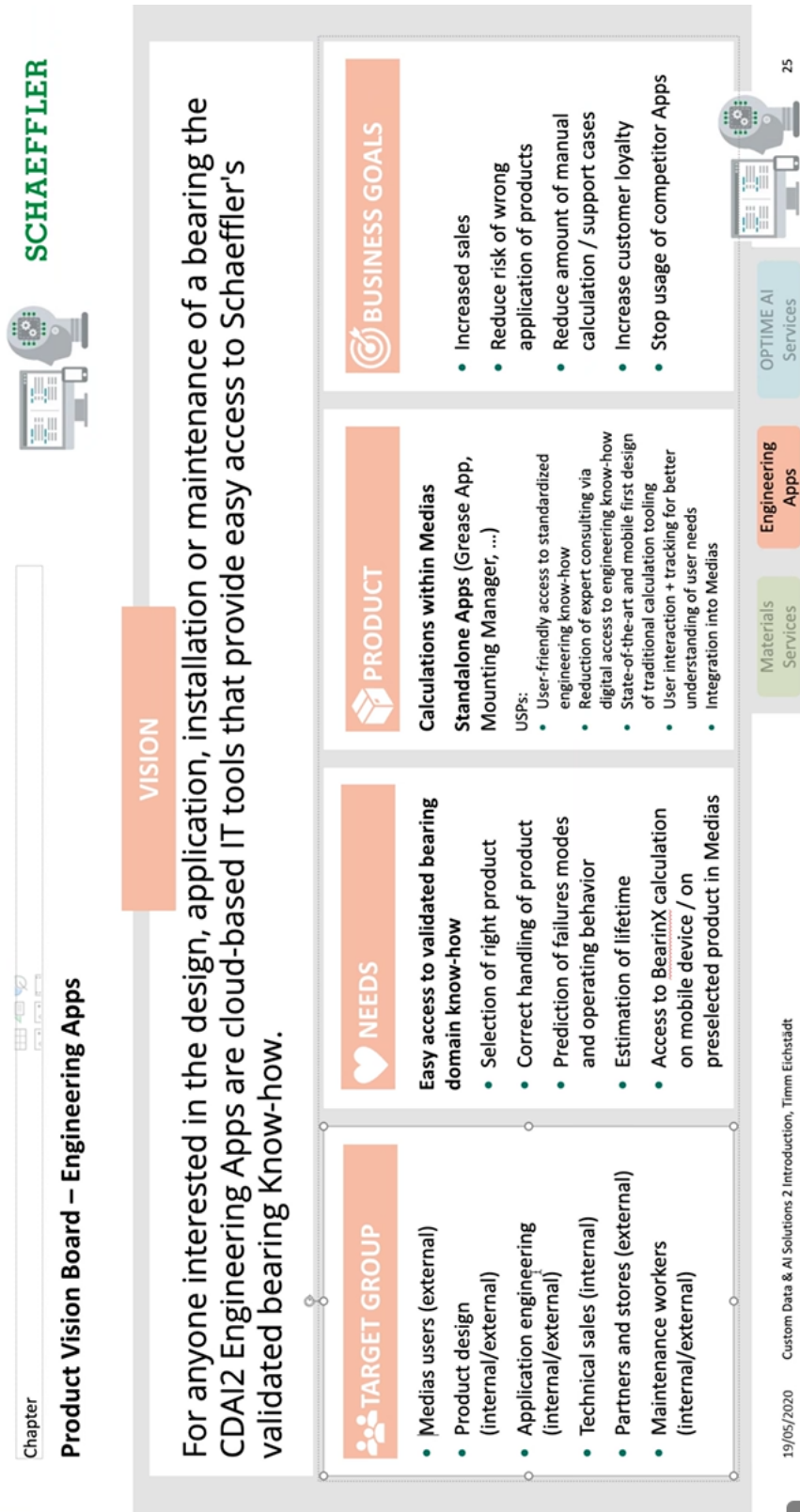
Wie bereits identifiziert, gibt es dennoch viele Weiterentwicklungsmöglichkeiten bei der Nutzung von Web Components bei Schaeffler. Die Engineering App verfügt noch nicht über alle gewünschten Funktionen, weshalb an dieser Stelle zweifelsfrei gesagt werden kann, dass dieses Projekt noch nicht beendet ist und die Web Component der Engineering App auch in Zukunft weiterentwickelt wird. Funktionen wie ein responsive Design und die Trennung der Styles müssen noch in die Engineering App eingearbeitet werden sowie noch viel Arbeit und Rechercheaufwand im Zusammenhang mit der User Tracking Thematik besteht, welche im Zuge einer weiteren Arbeit oder in der Abteilung erforscht werden kann.

Zusätzlich kamen auch bereits Ideen für neue Features auf, die dadurch bedingt sind, dass nun leichter Informationen zwischen der Web Component und der Medias Website ausgetauscht werden können. Eines dieser neuen Features wäre beispielsweise die Möglichkeit aus der Web Component heraus einen Button zu implementieren, der das ausgewählte Lager in den Warenkorb von Medias legt. Zudem muss auch noch eine Lösung gefunden werden, für den Zeitpunkt ab dem das Angular Feature „deploy-url“ verschwindet, da dieses wichtig für die Funktionalität der Web Component ist und noch kein passender Ersatz gefunden wurde.

Weiter ist nicht zu vergessen, dass die Engineering App nur eine der schaefflereigenen Apps ist, die zu einer Web Component umgebaut werden soll, weshalb davon auszugehen ist, dass sobald die Engineering App mit allen Funktionen verfügbar ist, auch die anderen Apps als Web Component nachgezogen werden sollen. Dies wird sowohl von der Abteilung als auch von Medias so gewünscht.

Deshalb ist davon auszugehen, dass dies erst der Start des Themas Web Component bei Schaeffler war, und auch in Zukunft in diese Richtung weiter recherchiert und entwickelt wird.

Anhang: Vision der Engineering Apps



Literatur

- [1] Adservio-Developer. *The Dark Side of Micro Frontends*. 11. Okt. 2021. URL: <https://www.adservio.fr/post/the-dark-side-of-micro-frontends> (besucht am 01.03.2023).
- [2] Alannah. *What Is Caching in Web Development?* 22. Aug. 2022. URL: <https://www.websitebuilderpoint.net/what-is-caching-in-web-development/> (besucht am 19.04.2023).
- [3] Robin Anderson. *Angular Build Output Hashing? The 20 Detailed Answer*. 12. Apr. 2023. URL: <https://brandiscrafts.com/angular-build-output-hashing-the-20-detailed-answer/> (besucht am 19.04.2023).
- [4] Angular-Build-Dokumentation. *ng build*. URL: <https://angular.io/cli/build> (besucht am 28.04.2023).
- [5] Angular-Dokumentation-Deployment. *Deployment*. 28. Feb. 2022. URL: <https://angular.io/guide/deployment> (besucht am 28.04.2023).
- [6] Angular-Dokumentation-Input-Parameter. *Sharing data between child and parent directives and components*. 28. Feb. 2022. URL: <https://angular.io/guide/inputs-outputs> (besucht am 28.04.2023).
- [7] Angular-Dokumentation-View-Encapsulation. *View Encapsulation*. 21. Apr. 2023. URL: <https://angular.io/guide/view-encapsulation> (besucht am 28.04.2023).
- [8] Atatus-Developer. *What is an iFrame? How does it work and what is it used for*. 6. Sep. 2022. URL: <https://www.atatus.com/blog/what-is-an-iframe> (besucht am 01.03.2023).
- [9] Bhargav Bachina. *6 Different Ways To Implement Micro Frontends With Angular*. 10. Feb. 2020. URL: <https://medium.com/bb-tutorials-and-thoughts/6-different-ways-to-implement-micro-frontends-with-angular-298bc8d79f6b> (besucht am 01.03.2023).
- [10] Bhargav Bachina. *How To Implement Micro-Frontend Architecture With Angular*. 10. Jan. 2020. URL: <https://medium.com/bb-tutorials-and-thoughts/how-to-implement-micro-frontend-architecture-with-angular-e6828a0a049c> (besucht am 01.03.2023).

- [11] Andrew Beng. *Ein modulares Web: iFrames und Web Components*. 11. Juli 2018. URL: <https://www.factorial.io/de/blog/iframes-und-web-components> (besucht am 01.03.2023).
- [12] Ahmed Bouchefra. *Building Web Components with Angular*. 29. Aug. 2020. URL: <https://buddy.works/tutorials/building-web-components-with-angular#transforming-the-angular-component-to-a-web-component> (besucht am 19.04.2023).
- [13] Jonathan Cowl. *Business Benefits and Advantages of Using Micro Frontends for Mobile*. URL: <https://ionic.io/resources/articles/business-benefits-of-micro-frontends-for-mobile> (besucht am 01.03.2023).
- [14] Frank Deges. *Grundlagen des E-Commerce: Strategien, Modelle, Instrumente*. 1. Aufl. Wiesbaden: Springer Gabler, 2020.
- [15] Nico Detmar. *Was ist ein iFrame? Einfach erklärt*. 29. Apr. 2019. URL: https://praxistipps.chip.de/was-ist-ein-iframe-einfach-erklart_110385 (besucht am 01.03.2023).
- [16] Justin Ellingwood. *Web Caching Basics: Terminology, HTTP Headers, and Caching Strategies*. 1. Apr. 2015. URL: <https://www.digitalocean.com/community/tutorials/web-caching-basics-terminology-http-headers-and-caching-strategies> (besucht am 19.04.2023).
- [17] Dominik Grosse Holtforth. *Schluesselfaktoren im E-Commerce: Innovationen, Skaleneffekte, Daten und Kundenzentrierung*. 1. Aufl. Wiesbaden: Springer Gabler, 2017.
- [18] Jenkins-Dokumentation. *Jenkins*. URL: <https://www.jenkins.io/> (besucht am 28.04.2023).
- [19] Yinyuan Liu. *Social Media in China: Wie deutsche Unternehmen soziale Medien im chinesischen Markt erfolgreich nutzen koennen*. 1. Aufl. Wiesbaden: Springer Gabler, 2016.
- [20] Medium-Developer-Guide. *HOW TO RUN ANGULAR BUILD LOCALLY*. 7. Apr. 2020. URL: <https://indepthcoder.medium.com/how-to-run-angular-build-locally-60108b1196bb> (besucht am 28.04.2023).
- [21] Microsoft-Build-Dokumentation. *Regular Expression Language - Quick Reference*. URL: <https://learn.microsoft.com/en-us/dotnet/standard/base-types/regular-expression-language-quick-reference> (besucht am 28.04.2023).

- [22] Will Morris. *What Is an iFrame? (And How to Use Them)*. 30. März 2020. URL: <https://www.elegantthemes.com/blog/wordpress/what-is-an-iframe-and-how-to-use-them> (besucht am 01.03.2023).
- [23] Mozilla-Developer-Guide-CORS-Guide. *Reason: CORS header 'Access-Control-Allow-Origin' missing*. 18. Apr. 2023. URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS/Errors/CORSMissingAllowOrigin> (besucht am 19.04.2023).
- [24] Mozilla-Developer-Guide-ShadowDom. *Using Shadow DOM*. 14. Apr. 2023. URL: https://developer.mozilla.org/en-US/docs/Web/API/Web_components/Using_shadow_DOM (besucht am 19.04.2023).
- [25] Mozilla-Development-Network. *From object to iframe-other embedding technologies*. 26. Feb. 2023. URL: https://developer.mozilla.org/en-US/docs/Learn/HTML/Multimedia%5C_and%5C_embedding%5C_other%5C_embedding%5C_technologies (besucht am 01.03.2023).
- [26] Carolina Ramirez. *Web Components. What are they? Pros, Cons and more*. 22. März 2022. URL: <https://medium.com/geekculture/web-components-what-are-they-pros-cons-and-more-77ad56711e49> (besucht am 01.03.2023).
- [27] Tailwind-Dokumentation. *Get started with Tailwind CSS*. URL: <https://tailwindcss.com/docs/installation> (besucht am 28.04.2023).
- [28] Tailwind-Dokumentation-Prefix. *Tailwind Configuration*. URL: <https://tailwindcss.com/docs/configuration> (besucht am 28.04.2023).
- [29] Armen Vardanyan. *Angular + Web Components: a complete guide*. 5. Feb. 2020. URL: <https://indepth.dev/posts/1116/angular-web-components-a-complete-guide> (besucht am 01.03.2023).
- [30] Webpack-Developer-Guide. *Webpack Concepts*. URL: <https://webpack.js.org/concepts/> (besucht am 19.04.2023).
- [31] Antonello Zanini. *5 Reasons You Should Adopt a Micro Frontend Architecture*. 8. Dez. 2021. URL: <https://www.sitepoint.com/micro-frontend-architecture-benefits/> (besucht am 01.03.2023).